arXiv:2509.25424v6 [cs.LG] 3 May 2026

Polychromic Objectives for Reinforcement Learning

Jubayer Ibn Hamid*, Ifdita Hasan Orney*, Ellen Xu, Chelsea Finn, Dorsa Sadigh

Stanford University

*Equal contribution. Correspondence to {jubayer, ifdi1101}@stanford.edu.

Abstract

Reinforcement learning fine-tuning (RLFT) is a dominant paradigm for improving pretrained policies for downstream tasks. These pretrained policies, trained on large datasets, produce generations with a broad range of promising but unrefined behaviors. Often, a critical failure mode of RLFT arises when policies lose this diversity and collapse into a handful of easily exploitable outputs. This convergence hinders exploration, which is essential for expanding the capabilities of the pretrained policy and for amplifying the benefits of test-time compute scaling. To address this, we introduce an objective for policy gradient methods that explicitly enforces the exploration and refinement of diverse generations, which we call a polychromic objective. We then show how proximal policy optimization (PPO) can be adapted to optimize this objective. Our method (1) employs vine sampling to collect on-policy rollouts and (2) modifies the advantage function to reflect the advantage under our new objective. Experiments on BabyAI, Minigrid, and Algorithmic Creativity show that our method improves success rates by reliably solving a larger set of environment configurations and generalizes better under large perturbations. Moreover, when given multiple attempts in pass@k experiments, the policy achieves substantially higher coverage, demonstrating its ability to maintain and exploit a diverse repertoire of strategies.

1 Introduction

Reinforcement learning fine-tuning (RLFT) is widely used to enhance the performance of pretrained models across diverse downstream domains. For instance, RLFT has been applied to steer large language models (LLMs) toward instruction following and complex reasoning (Ouyang et al., 2022; DeepSeek-AI et al., 2025; OpenAI et al., 2024; Qwen, 2025). A common thread across these settings is the availability of expressive generative models (i.e., pretrained distributions), trained on large and diverse datasets, that already exhibit a broad repertoire of strategies. RLFT then refines these distributions by reinforcing the strategies that yield higher reliability and performance.

However, exploration during RLFT remains a central challenge. Prior work (Cui et al., 2025; Zhao et al., 2025) has documented entropy collapse: instead of expanding their repertoire, fine-tuned policies concentrate probability mass on a narrow set of high-reward behaviors already present in the pretrained distribution, effectively sacrificing entropy and diversity. This limits exploration and prevents the discovery of alternative strategies that could expand the base model's capabilities. Empirically, this effect is captured by the pass@k metric, which measures the probability that at least one out of k independently sampled rollouts succeeds. When k is large, RL-fine-tuned models often underperform their pretrained counterparts because the latter retain greater diversity (Yue et al., 2025; Wu et al., 2025). Such diversity is practically important; it supports generalization to new tasks (Kumar et al., 2020) and amplifies test-time compute scaling (Snell et al., 2024).

The goal of this paper is to study how to induce policies to explore and refine a diverse repertoire of generations through RLFT. Our key insight is that algorithms should optimize objectives that explicitly encourage exploration and refinement of the diverse generations already embedded in the pretrained distribution. Standard regularization techniques, such as entropy bonuses, often induce local or token-level variation but fail to promote semantic or trajectory-level exploration and can be overshadowed by the RL objective. In contrast, we propose a unified formulation that

directly optimizes for a diverse set of successful behaviors, encouraging the policy to generate broad, varied trajectories rather than collapsing onto a few high-reward ones.

To this end, we propose set reinforcement learning, where the objective is defined over a set of trajectories sampled independently and evaluated by a multi-sample objective (Y. Tang et al., 2025). Unlike standard RL, which maximizes the likelihood of a single optimal trajectory, set RL maximizes the likelihood of an optimal set of trajectories under a set-level objective. Within this framework, we introduce polychromic objectives which combine reward and diversity by scoring sets highly only if they contain both successful and diverse trajectories. Optimizing a policy with respect to this objective is a principled approach towards encouraging the policy to explore and search for a diverse set of generations that also maximize reward. We then instantiate one such objective and show how proximal policy optimization (PPO) (Schulman, Wolski, et al., 2017) can be adapted to optimize it effectively, yielding a practical algorithm we call polychromic PPO. We evaluate our method on BabyAI (Chevalier-Boisvert et al., 2019), Minigrid (Chevalier-Boisvert et al., 2019), and Algorithmic Creativity (Nagarajan et al., 2025). Our results show that polychromic PPO achieves higher rewards and success rates, generates diverse trajectories that substantially improve pass@k coverage, and generalizes more robustly to perturbations in the initial state.

2 Preliminaries

We consider a Markov decision process defined by state space $\mathcal{S}$, action space $\mathcal{A}$, transition dynamics distribution $p(s_{t+1} \mid s_t, a_t)$, reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, initial state distribution ρ_0 and discount factor $\gamma \in (0, 1)$. In reinforcement learning (RL), the goal is to learn a policy that maximizes the value $V(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right] = \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\right]$ where $R(\tau)$ is the (discounted) sum of rewards in trajectory τ. The following, known as the performance difference lemma, is a useful result (Sham M. Kakade et al., 2002):

$$V_{\pi_\theta}(s_0) - V_{\pi_\beta}(s_0) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi_\theta}(\cdot|s_0), a\sim\pi_\theta(\cdot|s)}\left[A^{\pi_\beta}(s, a)\right]. \tag{1}$$

This shows that any update from policy π_β to policy π_θ such that, at all states visited by π_θ, the actions taken by π_θ yield positive advantage relative to π_β (i.e., $A^{\pi_\beta}(s, a) > 0$) will ensure that π_θ achieves strictly better performance i.e., $V(\pi_\theta) > V(\pi_\beta)$. One widely used RL algorithm is proximal policy optimization (PPO) (Schulman, Wolski, et al., 2017) which, iteratively, collects rollouts under a behavior policy π_β and updates a policy π_θ by constraining the divergence between the two policies from growing too large: letting $r_t = \pi_\theta(a_t \mid s_t)/\pi_\beta(a_t \mid s_t)$, PPO optimizes

$$\mathbb{E}_{s_t\sim d^{\pi_\beta}(\cdot), a_t\sim\pi_\beta(\cdot|s_t)}\left[\min\left(r_t\hat{A}(s_t, a_t), \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}(s_t, a_t)\right)\right] \tag{2}$$

where $d^{\pi_\beta}(s)$ is the stationary state-visitation distribution under policy π_β. Here, we use importance sampling to use actions sampled from π_β. To use states from the visitation distribution of π_β, we clip the ratios to keep the divergence small (Schulman, Levine, et al., 2017).

3 Reinforcing Exploration During RLFT

We aim to address the problem of entropy collapse during RLFT through a method that explicitly induces exploration by encouraging the generation of diverse trajectories. In §3.1, we introduce a variant of RL that allows for objectives beyond reward maximization and observe its various properties. This framework will provide us with a way to optimize objectives that are beyond return maximization, such as objectives that also encourage exploration. In §3.2, we specify the objective used within this framework for that purpose, which we call a polychromic objective. In §3.3, we propose our practical algorithm for optimizing the objective.

3.1 Set Reinforcement Learning

We introduce a variant of the standard RL setup in which, given an objective function, we optimize over a *set* of trajectories. We call this framework **set reinforcement learning** (set RL), where the goal is to solve the following optimization problem at a given state:

$$\max_{\theta} \mathbb{E}_{\tau_{1:n}\sim\pi_\theta(\cdot|s_0)} \left[f(s_0, \tau_1, \cdots, \tau_n) \right]. \tag{3}$$

Here $f(s_0, \tau_1, \cdots, \tau_n)$ is some objective function over trajectories $\tau_{1:n} = \{\tau_1, \cdots, \tau_n\}$ sampled independently from the policy. This is in contrast to standard RL where the problem, $\max_\theta \mathbb{E}_{\tau\sim\pi_\theta(\cdot|s_0)} [R(\tau)]$, uses an objective function, $R(\tau)$, defined over a single trajectory. Intuitively, algorithms optimizing Eq. (3) are optimizing for a set of trajectories. The generality of set RL makes it a powerful tool for objectives beyond sum of rewards. The defining feature of this setup is that, when optimizing Eq. (3) using policy gradient methods, all trajectories in the set $\tau_{1:n}$ must receive the same learning signal. Note that the objective can be optimized using policy gradient:

$$\begin{aligned}
&\nabla_\theta \mathbb{E}_{\tau_{1:n}\sim\pi_\theta(\cdot|s_0)}[f(s_0, \tau_{1:n})] \\
&= \mathbb{E}_{\tau_{1:n}\sim\pi_\theta(\cdot|s_0)} \Big[(f(s_0, \tau_{1:n}) - \hat{f}(s_0)) \sum_{i=1}^{n} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \Big]
\end{aligned} \tag{4}$$

where $\hat{f}(s_0)$ is a baseline for variance reduction. By definition, $\hat{f}(s_0)$ must be independent of the particular trajectories $\tau_{1:n}$ sampled inside the expectation. In this paper, we use the baseline $\hat{f}(s_0) = \mathbb{E}_{\tau_{1:n}\sim\pi_\theta(\cdot|s_0)} [f(s_0, \tau_{1:n})]$. The key feature of this estimator is that the advantage term $f(s_0, \tau_{1:n}) - \hat{f}(s_0)$ is shared across all trajectories in the set $\tau_{1:n}$. Therefore, the log-probability gradient of all trajectories in a set must be multiplied by the same factor, ensuring that all trajectories in the set receive the same learning signal. Formally, set RL is defined as problems for which there is no function $g(s_0, \tau)$, independent of θ, satisfying $\mathbb{E}_{\tau_{1:n}\sim\pi_\theta(\cdot|s_0)}[f(s_0, \tau_{1:n})] = \mathbb{E}_{\tau\sim\pi_\theta(\cdot|s_0)}[g(s_0, \tau)]$ for all policies π_θ.

This distinction is crucial. Note that an objective written over a set of trajectories does not necessarily define a new problem beyond standard RL. For example, if $f(s_0, \tau_{1:n}) = \frac{1}{n}\sum_{i=1}^{n} R(\tau_i)$, then $\mathbb{E}_{\tau_{1:n}\sim\pi_\theta(\cdot|s_0)}[f(s_0, \tau_{1:n})] = \mathbb{E}_{\tau\sim\pi_\theta(\cdot|s_0)}[R(\tau)]$, and the problem reduces exactly to standard RL. We use the term *set reinforcement learning* to refer specifically to objectives that are not reducible to any single-trajectory objective, i.e. problems that cannot be written as $\mathbb{E}_{\tau\sim\pi_\theta(\cdot|s_0)} [g(s_0, \tau)]$ for all θ where g is independent of θ. These are precisely the settings in which the value of a trajectory depends on the other trajectories, also sampled from π_θ, with which it is evaluated, making set-level optimization essential.

This setup contrasts with Y. Tang et al. (2025), which employs trajectory-specific baselines leading to leave-one-out advantages of the form $f(s_0, \tau_{1:n}) - f(s_0, \tau_{1:i-1}, \tau_{i+1:n})$; the update for trajectory τ_i depends on a baseline computed from the remaining trajectories, yielding individualized credit assignment. In our case, a uniform baseline provides a common update signal to all trajectories in the set, enabling optimization with respect to the quality of the entire *set* of trajectories. In other words, by definition, set RL does not distinguish between trajectories within a set, but instead optimizes the policy by comparing across sets as a whole. The framework allows for a broader class of objectives, such as inference-time objectives (Y. Tang et al., 2025) and objectives that induce exploration as we will show in §3.2.

Before we move on to our proposed algorithm, it is helpful to construct a notion of value functions in the framework of set reinforcement learning i.e., the expected sum of rewards as specified by the objective. We do so in a simplified (but impractical) setting. Suppose that at every state s encountered during an on-policy rollout, we can sample a set of n actions, $a_{1:n} \sim \pi_\theta(\cdot \mid s)$, which lead to n trajectories stemming out of every state that branch out at every timestep. Then, given this set of actions, $a_{1:n}$, taken from the state s, the policy gets the set reward $f(s, a_{1:n})$ with respect to the objective function f. Although such a setup is impractical for long-horizon tasks, analyzing it will help us better understand what set RL algorithms should aim to achieve.

Under this assumption, the data collection process naturally generates a state-visitation tree. Beginning at the root state s_0, each visited state s branches into n children, one for each sampled action. At depth t, the tree therefore contains n^t states, denoted by $s_t^{(1)}, \ldots, s_t^{(n^t)}$. For each state $s_t^{(i)}$, the corresponding set of n sampled actions is written as $(a_t)_{1:n}^{(i)}$. Given this tree-structured rollout and assuming an infinite-horizon discounted return, we define the value functions associated with set reinforcement learning as follows:

Definition 3.1 *Given a policy π generating a state-visitation tree and a set objective $f : \mathcal{S} \times \mathcal{A}^n \to \mathbb{R}$, the set value function $V^\sharp_\pi(s; f)$ and the set Q-function $Q^\sharp_\pi(s, a_{1:n}; f)$ are defined as*

$$V^\sharp_\pi(s; f) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \sum_{i=1}^{n^t} \gamma^t f(s_t^{(i)}, (a_t)_{1:n}^{(i)}) \Big| s_0 = s \right], \tag{5}$$

$$Q^\sharp_\pi(s, a_{1:n}; f) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \sum_{i=1}^{n^t} \gamma^t f(s_t^{(i)}, (a_t)_{1:n}^{(i)}) \Big|_{(a_0)_{1:n}=a_{1:n}}^{s_0=s,} \right] \tag{6}$$

We use the notation $V^\sharp$ and $Q^\sharp$ to distinguish it from the value function and Q-function in standard RL. Here, we assume that the discount factor $\gamma \in (0, \frac{1}{n})$ to ensure values remain bounded - the range is smaller since we add the expected sum of rewards from n actions stemming out of each state. Intuitively, $V^\sharp_\pi(s)$ is the expected discounted return of the entire state tree rooted at s (illustrated in Fig. 1), where the reward at each node is given by the set objective. In contrast, $Q^\sharp_\pi(s, a_{1:n})$ evaluates the expected return of the tree that begins at s with the specific action set $a_{1:n}$. Note that, although we assumed sets at the action-level instead of at the trajectory-level, this setting is equivalent to the trajectory-level set RL as in Eq. (3) under the objective function $F(s, \tau_{1:n^T}) := \sum_{t=0}^{T} \sum_{i=1}^{n^t} \gamma^t f(s_t^{(i)}, (a_t)_{1:n}^{(i)})$ in the finite-horizon setting (in the infinite horizon setting, we would have to take the limit $T \to \infty$).

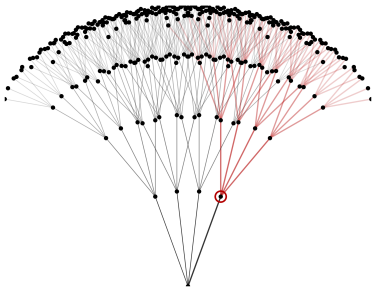


Figure 1: The set value of a state (circled) is the expected discounted return of the subtree (highlighted) rooted in this state.

These definitions help us better understand the objective of set reinforcement learning. In standard reinforcement learning, we want to learn the policy such that the expected return from a trajectory is maximized. In set reinforcement learning, we want to learn the policy that maximizes the expected return from a *tree* generated using our policy is maximized. Given these definitions, we have the following result which is an extension of the performance difference lemma (Sham M. Kakade et al., 2002) to the set reinforcement learning setting (see Section C.1 for proof):

Lemma 3.2 *Given any two policies π_θ and π_β and a fixed initial state s_0, under any set objective function f,*

$$V^\sharp_{\pi_\theta}(s_0; f) - V^\sharp_{\pi_\beta}(s_0; f) = \frac{1}{1 - \gamma n} \mathbb{E}_{s \sim d^\sharp_{\pi_\theta}(\cdot), a_{1:n} \sim \pi_\theta(\cdot|s)} \left[A^\sharp_{\pi_\beta}(s, a_{1:n}; f) \right].$$

Here $d^\sharp_\pi(s)$ is the stationary state-visitation distribution induced by π_θ. Similar to the standard reinforcement learning setting, this result says that if we update the policy π_β to π_θ such that, at all states visited by π_θ, the advantage $A^\sharp_{\pi_\beta}(s, a_{1:n}; f) = Q^\sharp_{\pi_\beta}(s, a_{1:n}; f) - V^\sharp_{\pi_\beta}(s)$ of a set of actions taken by our new policy π_θ is positive, then we will get a policy that has strictly higher performance (as measured by its value). This suggests that many of the principles underlying methods like PPO can be extended to the set reinforcement learning paradigm as well for policy improvement.

Having seen the generality of set reinforcement learning, we now construct an objective function to be used within this framework that will allow us to induce exploration.

3.2 A Practical Polychromic Objective

Our central construction is the notion of polychromic objectives that are aimed towards training a policy to explore and learn a diverse set of behaviors. Intuitively, these are set objective functions $f_{\text{poly}} : \mathcal{S} \times \mathcal{T}^n \to \mathbb{R}$ that jointly capture (1) the success of a set of trajectories in terms of reward and (2) the degree to which the set exhibits exploration or diversity. While we later generalize this construction in §5, in this section we focus on the specific instance used in our algorithm and experiments:

$$f_{\text{poly}}(s, \tau_{1:n}) := \frac{1}{n} \sum_{i=1}^{n} R(\tau_i) d(s, \tau_{1:n}), \tag{7}$$

where $R(\tau_i)$ is the discounted sum of rewards in trajectory τ_i, and $d(s, \tau_{1:n})$ is a function that quantifies the diversity of trajectories within the set. We require that both $R(\tau_i)$ and $d(s, \tau_{1:n})$ are normalized between 0 and 1.

Because the set-RL gradient uses a shared advantage for all trajectories in a set, this objective increases the likelihood of successful behaviors *and* diverse exploratory trajectories. Unlike prior approaches, the shared advantage term amplifies exploratory trajectories that do not (yet) yield high rewards, pushing the policy to discover diverse strategies.

Various diversity metrics have been studied and incorporated in reward functions in prior works; examples include the Vendi Score (Friedman et al., 2023) and classifier-guided diversity (Zhang et al., 2025; Li et al., 2025). Our algorithm is designed to be agnostic to the choice of metric: given any diversity function, we evaluate the success and diversity of a set of trajectories and optimize the policy to maximize both in sets.

3.3 Polychromic PPO

In this section, we present an algorithm for optimizing Eq. (7) by modifying PPO, which is motivated by the extension of the performance difference lemma to the set reinforcement learning framework (as shown in Lemma 3.2). Our approach differs from standard PPO in two key respects: the method using which we sample on-policy rollouts and the advantage function used in the update.

A direct implementation of the definition of set advantage functions as in Lemma 3.2 would require sampling n actions from every visited state, leading to exponential data requirements. To avoid this, we instead rely on vine sampling (Schulman, Levine, et al., 2017; Kazemnejad et al., 2025) for on-policy data collection. In vine sampling, after collecting an initial set of rollouts, we select a subset $\{s_1, \ldots, s_p\}$ of the states visited, called rollout states. At each rollout state s_i, we generate N additional rollouts starting from s_i. This procedure ensures that we obtain multiple states with independently sampled trajectory sets stemming out of them. The particular scheme we use is closely related to vine TRPO (Schulman, Levine, et al., 2017); details are deferred to §A.1.1. Our algorithm, however, is compatible with any vine-sampling method that guarantees sufficient vine coverage. Note that, since vine sampling requires the ability to reset the environment, our algorithm is only applicable to environments where such resets are possible.

Given access to sets of trajectories from each rollout state, we can estimate the polychromic value functions and, in turn, construct a practical polychromic advantage estimator. At a rollout state s_t from which we generated $N > n$ trajectories, we estimate the polychromic advantage as

$$A^\sharp(s_t, a_t; f_{\text{poly}}) = \frac{1}{n} \sum_{i=1}^{n} R(\tau_i) d(s_t, \tau_{1:n}) - \hat{V}^\sharp(s_t; f_{\text{poly}})$$

where $a_t \in \tau_i$ for some $i \in 1, \ldots, n$. Since PPO requires an advantage defined for individual actions, we assign to each action a_t the advantage of the set $\tau_{1:n}$ that contains it. In other words, given a set of trajectories $\tau_{1:n}$, all actions taken from s_t in this set receive the same update signal as desired in set reinforcement learning. We use the following Monte Carlo estimate of the value baseline: $V^\sharp(s_t; f_{\text{poly}}) = \frac{1}{M} \sum_{i=1}^{M} f_{\text{poly}}(s_t, \tau_{1:n}^{(i)})$, where $\tau_{1:n}^{(i)}, i \in \{1, \cdots, M\}$ denotes the M independently sampled sets of n trajectories starting from s_t. This unbiased estimate was sufficient for our experiments, but one can trade off variance further by using biased estimates which we leave to future work.

For non-rollout states, the update remains the same as standard PPO, using generalized advantage estimation (GAE) (Schulman, Moritz, et al., 2018). As is often used in practical implementations of PPO, we also include a per-state KL penalty $D_{\text{KL}}\big(\pi_\beta(\cdot \mid s) | \pi_\theta(\cdot \mid s)\big)$ at every state visited, which we found helpful for stability. The pseudocode is presented in Algorithm 1, with modifications relative to PPO highlighted; extended pseudocode and implementation details are given in Section A. We report all hyperparameters in §A.1.2. Note that the size of sets n used in set RL and the number of trajectories N generated from rollout states are fixed hyperparameters (in all our experiments, we used $n = 4$ and $N = 8$).

Note that vine sampling is not a necessary component of the algorithm. Consider an environment with a fixed initial state. In this case, if we sample rollouts independently without any vine sampling, we will naturally get multiple rollouts from the fixed initial state allowing us to apply the polychromic advantage to the initial state. Vine sampling simply enables us to apply the polychromic advantage to a larger number of states (i.e., all the rollout states) which helps encourage exploration at various stages of a rollout.

Algorithm 1 Polychromic PPO

for iteration $= 1, 2, \ldots$ **do**
  Collect trajectories under π_β; rollout vines $\tau_{1:N}$ from rollout states
  if s_t rollout state **then**
    Form sets, $g_1, \ldots, g_M$ of n trajectories from s_t
    Set $\hat{A}(s_t, a_t) = f_{\text{poly}}(s_t, g_i) - \frac{1}{M}\sum_{j=1}^{M} f_{\text{poly}}(s_t, g_j)$ for $(s_t, a_t) \in g_i$
  else
    Compute $\hat{A}(s_t, a_t)$ via GAE
  end if
  Update π_θ for K epochs on minibatches $\mathcal{B}$ by maximizing the PPO objective in Eq. (2)
  Set $\pi_\beta \leftarrow \pi_\theta$
end for

4 Experimental Evaluation

Our experiments aim to answer two questions: (1) How does polychromic PPO, a set RL algorithm that explicitly encourages diverse trajectory generation, affect performance? More specifically, does improving diversity come at a significant cost in accuracy and success rate? (2) Does polychromic PPO encourage the policy to explore and learn diverse behaviors? In particular, learning to solve a task through diverse generations should, ideally, increase the pass@k performance i.e., the probability of succeeding at least once when given multiple attempts. (3) Does encouraging the policy to explore and maximize the diversity of generated trajectories help the policy to be more robust to perturbations in the state-visitation distribution? To address these questions, we evaluate on Minigrid (Chevalier-Boisvert et al., 2019), BabyAI (Chevalier-Boisvert et al., 2019), and Algorithmic Creativity (Nagarajan et al., 2025).

Minigrid and BabyAI are grid-world platforms with multiple rooms connected by locked doors and populated with keys, balls, and distractors. Agents receive natural language goals ranging from simple (for example, *go to the red ball*) to highly compositional (for example, *open a red door and then go to the ball on your left after placing the grey ball next to a door*). We use a language-conditioned convolutional neural network (CNN) policy that is pretrained on expert demonstrations provided in the official datasets (Chevalier-Boisvert et al., 2019). We then fine-tune and evaluate on 50 fixed configurations (each configuration specifies a grid layout and mission pair).

Algorithmic Creativity is a set of tasks for quantifying model creativity (Nagarajan et al., 2025). In the *triangle discovery* task, an agent must autoregressively output sequences of triangles from undirected graphs that contain many triangles. Solving the task in each graph requires recall and combinations of past knowledge, as the graph is not revealed in-context, but learned during pretraining and through interactions in RLFT. We pretrain on data drawn from 10 graphs and fine-tune on 3 graphs. Here, the verifiable reward is whether the generated sequence is a valid triangle in the graph.

We compare polychromic PPO (Poly-PPO) with REINFORCE with baseline (Williams, 1992) and standard PPO (Schulman, Wolski, et al., 2017). Furthermore, we compare with a UCB-style regularization (Azar et al., 2017) where we add $\lambda_{\text{UCB}} \cdot \min\{1, N(s,a)^{-\frac{1}{2}}\}$ to every advantage $\hat{A}(s,a)$. Here, $N(s,a)$ is the number of times that action a was sampled from state s and λ_{UCB} is a hyperparameter we tune for each method. For polychromic PPO, we define the diversity function $d(s, \tau_{1:n})$ to be the fraction of semantically distinct trajectories in $\tau_{1:n}$; in Minigrid/BabyAI, two trajectories are called distinct if they visit different sets of rooms and, in Algorithmic Creativity, two trajectories are distinct if they visit different sets of nodes. In both cases, $d = 0$ if all trajectories visit the same set of rooms or nodes. More details on the environment and our implementation can be found in §A. Throughout all settings, we generate $N = 8$ trajectories at rollouts states for vine sampling and use sets of size $n = 4$ for poly-PPO (see Algorithm 1).

All environments we evaluate on test on tasks that are long-horizon, often requiring sequential plans that change, and sparse reward. Consequently, fine-tuning requires an RL method that can both explore and exploit. Since the pretrained policy struggles to solve several tasks, the policy must strategically explore during RL fine-tuning, and to succeed across all evaluation tasks, the policy must avoid collapsing onto behaviors that solve only a subset while failing to generalize to the rest.

Environment	Pretrained policy	REINFORCE	REINFORCE w/ UCB	PPO	PPO w/ UCB	Poly-PPO (ours)	Poly-PPO w/ UCB (ours)
Goto	(0.246, 34.2)	(0.533, 73.0)	(0.538, 73.4)	(0.406, 46.2)	(0.428, 47.4)	**(0.575, 80.2)**	**(0.561, 76.2)**
Pickup	(0.141, 21.4)	(0.259, 39.8)	(0.391, 56.0)	(0.283, 33.4)	(0.243, 27.8)	**(0.452, 63.2)**	**(0.486, 65.6)**
Synthseq	(0.157, 20.2)	**(0.325, 45.4)**	**(0.361, 47.8)**	(0.277, 32.2)	(0.224, 26.2)	**(0.341, 47.0)**	**(0.317, 43.2)**
Bosslevel	(0.212, 20.6)	(0.266, 33.4)	(0.286, 36.4)	(0.336, 38.8)	(0.310, 35.8)	**(0.378, 45.2)**	**(0.379, 46.8)**
Four Rooms	(0.469, 70.4)	(0.639, 89.6)	**(0.672, 92.6)**	**(0.618, 89.2)**	(0.502, 78.6)	**(0.666, 92.4)**	**(0.653, 91.8)**

Table 1: Performance on BabyAI and MiniGrid tasks. Each entry is reported as (x, y), where x is the average episodic reward and y is the success rate (in %). Results are averaged over 100 rollouts across 50 configurations and 3 random seeds.

4.1 How Does Polychromic PPO Affect Performance?

The results summarizing performance, in terms of average reward and success rate, on Minigrid and BabyAI are provided in Table 1. Pretrained policies are noisy, achieving low success rates; policies that explore effectively end up maximizing both success rate and coverage of environments it can solve. We find that polychromic PPO consistently matches or outperforms the best baseline in reward and success. Adding the UCB bonus helps the baselines, REINFORCE and PPO, improve performance in some environments. However, UCB is complementary to polychromic PPO as well - the bonus enables the agent to achieve higher performance in *Pickup* and *Bosslevel*.

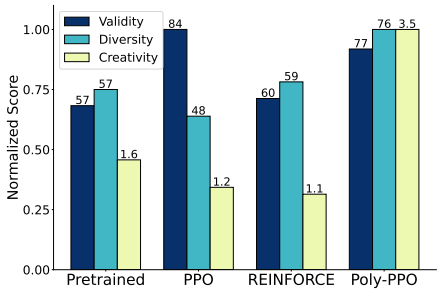


Figure 2: Results on Algorithmic Creativity. Bars show normalized values for each metric, with raw values above each bar.

Results for Algorithmic Creativity are summarized in Fig. 2. We rollout 100 times across the 3 graphs seen during RLFT. The creativity metric is defined as the percentage of generations that are unique, valid triangles that were not seen in the pretraining data (Nagarajan et al., 2025). We also report validity (number of valid triangles constructed), and diversity (unique number of valid triangles). PPO substantially increase validity compared to the pretrained policy, but lower creativity and diversity. On the other hand, polychromic PPO achieves slightly lower validity than standard PPO, but the gap is modest and highlights the trade-off it strikes between success and exploration which we discuss in the next subsection.

4.2 Does Polychromic PPO Encourage Diverse Trajectories?

Success rates do not adequately represent the coverage of tasks that the policy can solve. Since success rates are averaged across all configurations, a policy that overfits to a subset may achieve high reward there while failing elsewhere. To probe this, we examine pass@k curves - for each configuration, we provide the policy k attempts and find the fraction of configurations the policy can solve, which is called the pass rate. As k increases, methods that generate diverse (but effective) trajectories should achieve higher coverage. Moreover, because all configurations were seen during training, every method should ideally surpass the pretrained policy at all k. When this fails to occur, it suggests that RL fine-tuning has caused the policy to forget behaviors useful for some configurations, overfitting instead to others.

Pass@k results on the BabyAI environments are shown in Fig. 3. We first discuss all methods without the UCB bonus. We observe that REINFORCE does not improve pass rate sufficiently fast as the number of attempts increases: despite higher success rates overall, its coverage is lower than the pretrained policy at large k. PPO, on the other hand, starts off from a much lower pass rate than other methods at small k but the pass rate increases as k grows; however, it is still lower than the pretrained policy and significantly lower than Poly-PPO. This indicates that these baseline methods suffer from an inherent trade-off between diversity and accuracy in the generations. In comparison, Poly-PPO achieves substantially higher pass rate than all baselines. It also achieves equal or higher pass rate than the

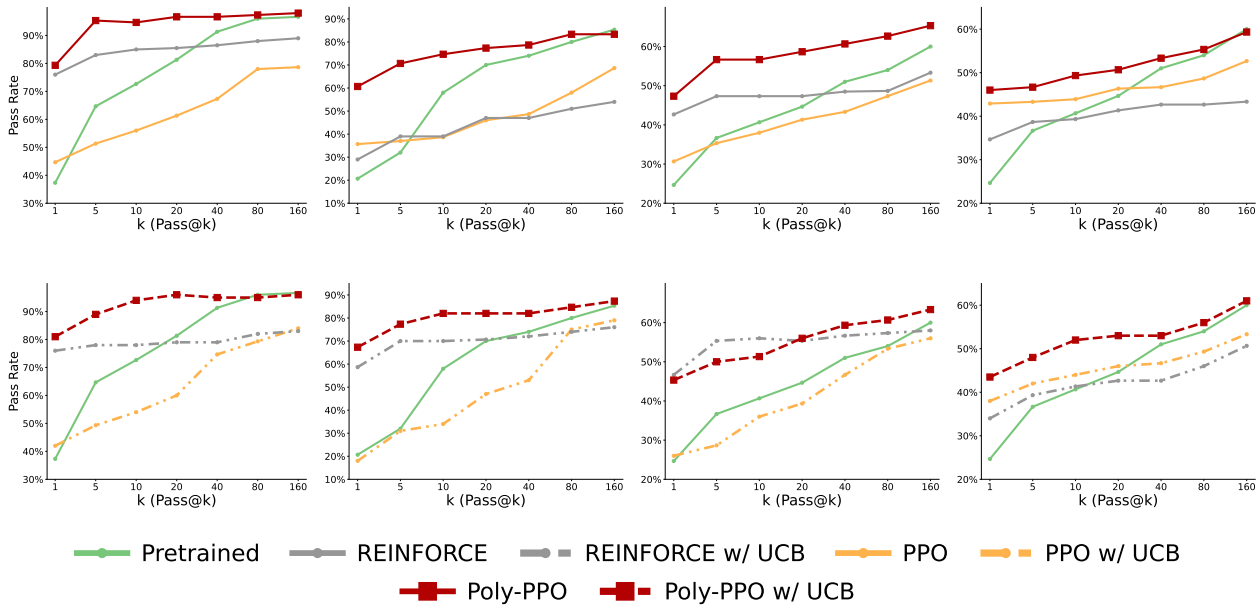


Figure 3: Pass@k on BabyAI tasks. Top: methods without UCB. Bottom: methods with UCB. Columns show Goto, Pickup, Synthseq, and Bosslevel. Each curve is pass rate vs. number of attempts.

pretrained policy at almost all values of k. Another indicator for the higher diversity in generations is that the pass rate for Poly-PPO continues to rise until about $k = 80$, whereas the baselines saturate much earlier (around $k = 20$). The effect is pronounced, especially, in *Bosslevel* where Poly-PPO achieves around 15% higher pass rate as k grows to 160 whereas baselines see modest increase.

We next examine the effect of adding the UCB bonus. For REINFORCE, the bonus improves pass rate at small k in *Pickup* and *Synthseq*, but the gain saturates around $k = 10$ and UCB has no effect in *Bosslevel* or *Goto*. For PPO, the bonus reduces pass rate at small k, and although it improves performance at larger k, the gap to the pretrained policy and Poly-PPO remains. By contrast, combining UCB with Poly-PPO yields equal or higher coverage across most k (except small k in *Synthseq*), showing that Poly-PPO preserves and refines pretrained diversity rather than collapsing onto narrow behaviors.

In the *Triangle Discovery* task, we find that polychromic PPO achieves substantially higher diversity and creativity. In particular, Poly-PPO outperforms all baselines, including the pretrained policy, on both diversity and creativity metrics, as shown in Fig. 2. Although it achieves slightly lower validity than PPO (significantly larger than REINFORCE though), Poly-PPO encourages broad exploration and the discovery of novel solutions. This trend is further reflected in the pass@k evaluation (see Fig. 4). Specifically, validity pass@k evaluates pass rate in k attempts where the agent passes if it generates a valid triangle in k attempts. On the other hand, creativity pass@k evaluates pass rate where the agent passes if it generates at least one creative triangle (unique triangle not seen in pretraining data) in k attempts. Finally, diff@k quantifies the number of unique triangles obtained in k attempts. We find that Poly-PPO outperforms baselines in both creativity and diversity metrics, while attaining greater validity performance than the pretrained policy. Notably, even though REINFORCE achieves high diversity, it comes at the significant cost in validity@1 where it is even below the pretrained policy.

4.3 Does polychromic PPO generalize to state perturbations?

We evaluate generalization under perturbed initial states in Minigrid and BabyAI. For each grid–mission configuration, we first find all the rooms visited by the pretrained policy under high-temperature sampling over 100 rollouts. Then, we select 10 states randomly from inside each room as our initial states. Effectively, this changes the initial state to a completely different room in such a manner that the task remains solvable from the new initial state. Note that

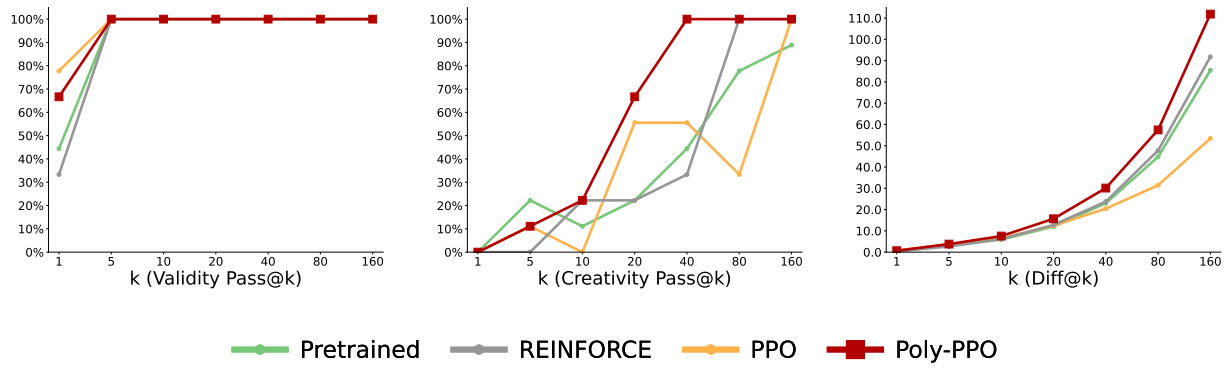


Figure 4: Pass@k results on Algorithmic Creativity. For validity pass@k and creativity pass@k, the agent gets a pass if at least one of the k attempts was a valid and creative triangle, respectively. In diff@k evaluation, we evaluate the number of generations that were unique given k attempts.

Environment	Pretrained policy	REINFORCE w/ Baseline	REINFORCE w/ UCB	PPO	PPO w/ UCB	Polychromic PPO	Poly-PPO w/ UCB
Goto	30.2	41.3	37.1	21.1	18.4	**60.6**	54.3
Pickup	15.2	22.0	20.5	12.5	8.87	**33.4**	28.0
Synthseq	20.0	19.3	26.2	16.6	11.5	**30.6**	**32.1**
Bosslevel	23.8	22.5	27.6	26.6	28.2	**34.3**	**32.8**
Four Rooms	65.0	82.7	81.5	15.3	14.2	**88.7**	**87.2**

Table 2: Average pass rate (%) in one attempt on BabyAI and Minigrid tasks under large initial-state perturbations.

this randomization substantially changes the task; as shown in Fig. 6, with respect to the new initial state, a successful trajectory would require very different strategies. From the new start state, we evaluate using pass@1 for all states in each layout. As shown in Table 2, consistently, polychromic PPO generalizes more reliably than the baselines under these perturbations.

5 Entropy Analysis

In this section, we analyze how the entropy of a policy evolves when trained to optimize the polychromic objective in Eq. (7). Our guiding question is: Under the polychromic objective, on which actions is a policy most likely to collapse its probability mass? In our analysis, we restrict our attention to the setting with time horizon $H = 1$, with binary rewards. We assume a discrete action space and that the diversity function $d(s, a_{1:n})$ equals the fraction of actions in $a_{1:n}$ that are distinct ($d = 0$ if the set is a singleton). We assume that our policy has a softmax parameterization. Before turning to the polychromic objective itself, we extend the entropy analysis of Cui et al. (2025) to the set RL setting. Given any set objective $f : \mathcal{S} \times \mathcal{A}^n \to \mathbb{R}$, we ask: how does the entropy of the policy change after one-step update (from π_θ^k to π_θ^{k+1}) when learning under this set-RL framework? The following proposition characterizes the first-order change:

Proposition 5.1 *Consider the set RL setup in state s. After one update to the policy, the change in entropy, $\Delta = \mathcal{H}\left(\pi_\theta^{k+1} \mid s\right) - \mathcal{H}\left(\pi_\theta^{k} \mid s\right)$, is given by*

$$\Delta \approx -\alpha \mathrm{Cov}_{a_{1:n}}\left(\frac{1}{n}\sum_{i=1}^{n} \log \pi_\theta^k(a_i \mid s), \mathrm{Cov}_{a'_{1:n}}\left(f(s, a'_{1:n}), \sum_{i,j=1}^{n} 1\{a_i = a'_j\}\right)\right),$$

where both covariances are taken with respect to $\pi_\theta^k(\cdot \mid s)$ and α is the learning rate.

The proof can be found in §C. This result provides a lens for understanding when and where entropy collapse occurs. Intuitively, suppose that for some reference set $a_{1:n}$ there is a strong covariance between (i) the overlap of $a_{1:n}$ with the sampled sets and (ii) the value of the objective f. As the policy concentrates more probability mass on such sets $a_{1:n}$, the entropy decreases. Conversely, when the covariance is strongly negative, the policy reallocates probability mass to sets with higher value under f, which increases entropy. Importantly, although set RL evaluates entire sets without attributing credit to individual trajectories, entropy collapse around a particular set still requires that its constituent trajectories, in expectation, contribute to raising the value of sampled sets overall.

Thus, the central question becomes: Which sets $a_{1:n}$ are most prone to entropy collapse under the polychromic objective? Our analysis proceeds by introducing and studying the following key object:

Definition 5.2 *The scaffold value of a set of actions, $a_{1:n}$, under a policy π and a set-RL objective $f : \mathcal{S} \times \mathcal{A}^n \to \mathbb{R}$ is defined to be*

$$\Lambda_f(a_{1:n}; \pi) := \mathrm{Cov}_{a'_{1:n} \sim \pi(\cdot|s)}(f(s, a'_{1:n}), \frac{1}{I(a_{1:n})} \sum_{i,j=1}^{n} 1\{a'_i = a_j\})$$

where $I(a_{1:n})$ is the maximum size of the intersection of $a_{1:n}$ with any other set $a'_{1:n}$. We refer to the space $\Lambda_f(\pi) = \{(a_{1:n}, \Lambda_f(a_{1:n}; \pi) \mid a_{1:n} \in \mathcal{A}^n\}$ as the scaffold of the policy π.

The scaffold represents, for every action set $a_{1:n}$, a measure of the policy's propensity to collapse its entropy around that set. We illustrate this further through the following lemma which shows us how the scaffold value of an action set affects the change in the probability of a policy sampling the set (proof in §C.3):

Lemma 5.3 *Consider any set of actions $a_{1:n} \in \mathcal{A}^n$. The change in the log probability of sampling this set of actions after one policy update using set RL can be written as the following first-order approximation:*

$$\log \pi_\theta^{k+1}(a_{1:n} \mid s) \approx \log \pi_\theta^k(a_{1:n} \mid s) + \lambda \Lambda_f(a_{1:n}; \pi_\theta^k) - \lambda' C(\theta^k)$$

where $C(\theta^k)$ is a function independent of $a_{1:n}$, and λ and λ' are constants.

With this apparatus in hand, we next show that the polychromic objective rules out collapse onto homogeneous sets of actions (proof in §C.4):

Proposition 5.4 *Consider the polychromic objective in Eq. (7). For any homogeneous set $a_{1:n} = \{a\}$ where $r(s, a) = 1$, there exists $\epsilon \in (0, 1)$ such that $\Lambda_{f_{\text{poly}}}(a) < 0$ when $\pi_\theta(a \mid s) > \epsilon$. Furthermore, the scaffold values of these homogeneous sets satisfy the bound $\Lambda_{f_{\text{poly}}}(a) \leq \sqrt{\frac{p(1-p)}{n}}$.*

This result shows that once a successful action a accumulates sufficient probability mass, the polychromic objective automatically prevents further entropy collapse onto sets that only contain this action and, when we use larger sets in the set RL framework, the upper bound on the scaffold value of a homogeneous set decreases. This is desirable since it suggests that our policy learns to generate this action without incessantly collapsing probability mass on such homogeneous sets.

Next, we establish that the scaffold values of heterogeneous sets with successful actions are attractors of probability mass (proof in §C.5):

Proposition 5.5 *Suppose $a_{1:n}$ is heterogeneous where each a_i is unique with probability $\frac{p}{n}$ where $p \in (0, \frac{1}{n})$. Suppose exactly q of the n actions satisfy $r(s, a_i) = 1$, and that any other action $a' \notin a_{1:n}$ with $\pi_\theta(a' \mid s) > 0$ yields $r(s, a') = 0$. Then, the scaffold value of $a_{1:n}$ satisfies $\Lambda_{f_{\text{poly}}}(a_{1:n}) > \frac{qp^n(1-p)}{n} \left(1 - \left(1 - \frac{1}{n}\right)^n\right)$.*

Note that the set in this proposition includes unsuccessful actions as well that contribute diversity. As such, there are, likely, several such heterogeneous sets (provided that the size of sets n is large enough) with positive scaffold values that attract more probability mass than homogeneous sets. Furthermore, the lower bound guarantee increases as the number of successful actions in the set increases. The polychromic objective therefore channels entropy collapse toward those sets that balance success and exploration, rather than permitting collapse onto homogeneous behaviors.

These results motivate our general construction of polychromic objectives. Broadly, such objectives reward both (i) the returns achieved by a set of trajectories and (ii) the diversity of trajectories in the set.

Definition 5.6 *A polychromic objective is a function*

$$\varphi : \mathcal{S} \times \mathcal{T}^n \to \mathbb{R}$$

that factors as

$$\varphi(s, \tau_{1:n}) = \varphi^{(R)}(s, \tau_{1:n})\varphi^{(d)}(s, \tau_{1:n}),$$

where $\varphi^{(R)}$ and $\varphi^{(d)}$ are scalar-valued functions satisfying:

1. $\mathrm{Cov}_{\tau_{1:n}\sim\pi_\theta(\cdot|s)}(\varphi^{(R)}(s, \tau_{1:n}), \sum_{i=1}^n R(\tau_i)) > 0$,
2. $\mathrm{Cov}_{\tau_{1:n}\sim\pi_\theta(\cdot|s)}\left(\varphi^{(d)}(s, \tau_{1:n}), \sum_{i=1}^n 1\{\tau_i = \tau\}\right) < 0$ *for any τ, and*
3. $\varphi^{(R)}(s, \cdot)$ *and* $\varphi^{(d)}(s, \cdot)$ *share the same range, i.e.,*

$$\inf \varphi^{(R)}(s, \cdot) = \inf \varphi^{(d)}(s, \cdot), \quad \sup \varphi^{(R)}(s, \cdot) = \sup \varphi^{(d)}(s, \cdot),$$

where the infimum and supremum are taken over all sets $\tau_{1:n}$.

In other words, a polychromic objective factors into a reward component with positive covariance to return and a diversity component with negative covariance to homogeneity such that both components have equal range. Their product enforces that both success and diversity are indispensable: entropy collapse is guided away from trivial memorization and toward exploratory policies.

6 Related Work

Policy gradient methods (Sutton, McAllester, et al., 1999; Sutton and Barto, 2018; Sham M Kakade, 2001) are a widely used family of algorithms for online reinforcement learning. Numerous variants have been developed to reduce variance and improve sample efficiency (Bhatnagar et al., 2009; Degris et al., 2013; Lillicrap et al., 2016; Wang et al., 2017; Schulman, Levine, et al., 2017; Schulman, Wolski, et al., 2017; Roux et al., 2025). More recently, in the context of LLM fine-tuning, several works have replaced the learned critic with empirical estimates (DeepSeek-AI et al., 2025; Yu et al., 2025; Zheng et al., 2025; Liu et al., 2025). The availability of test-time compute has motivated training objectives aligned with inference-time metrics (Y. Tang et al., 2025; Chow et al., 2024) and algorithms bridging reinforcement learning with maximum likelihood estimation (Tajwar, Zeng, et al., 2026).

RL fine-tuning often suffers from entropy collapse, where policies concentrate on a few high-reward behaviors and lose coverage of the pretrained distribution (Cui et al., 2025; Wu et al., 2025; Yue et al., 2025). Entropy bonuses (Haarnoja, H. Tang, et al., 2017; Haarnoja, Zhou, et al., 2018; Schulman, Wolski, et al., 2017; Seo et al., 2021; Islam et al., 2019) mitigate this locally but may struggle to induce semantic or trajectory-level exploration. In contrast, UCB-style bonus (He et al., 2025; Lanchantin et al., 2025) can be very effective in encouraging exploration especially earlier on during RLFT; however, the objective does not explicitly encourage preserving this exploratory behavior or diversity throughout training as the UCB bonus decays over time. Covariance controls (Cui et al., 2025) can slow down entropy collapse but it is not clear that these methods necessarily encourage the policy to explore diverse generations. Our analysis builds on top of the insights Cui et al. (2025) propose and we show that our method also exercises covariance controls on homogeneous behaviors. Intrinsic curiosity (Pathak et al., 2017) also encourage exploration but it is not clear that these methods either scale to high-dimensional spaces or solve the local versus semantic exploration problem. Other approaches, more similar to our work, explicitly reward diversity, e.g., diversity-weighted objectives (Li et al., 2025), outcome-based UCB bonus (Song et al., 2025) or train explicitly for exploration using synthetic data (Tajwar, Jiang, et al., 2025). The crucial distinction our work makes is the set reinforcement learning framework that enables learning a set of behaviors wherein some trajectories receive positive gradient updates for exploration despite not contributing rewards to the set.

7 Conclusion

We framed the problem of learning a diverse set of successful behaviors in terms of set reinforcement learning and proposed optimizing the polychromic objective, which evaluates sets of actions using both reward and diversity. We then designed our algorithm polychromic PPO, a variant of PPO that incorporates vine sampling and a modified advantage estimator to optimize this objective. There are several limitations of our approach. Our approach requires the ability to reset the environment to any set to enable vine sampling. Otherwise, we require the MDP to have sufficiently deterministic transitions such that replaying actions can lead us approximately to the reset state. Moreover, ensuring sufficient vine coverage can be challenging, especially for very long-horizon tasks. The experiments in this paper are in settings where we could easily design a diversity function; however, designing or learning diversity functions can be difficult in various settings like for continuous control. Throughout this paper, we relied on Monte Carlo estimates to compute the advantage which could be high variance in other settings. Future work could develop more efficient estimators, or adopt curriculum and annealing schemes that balance exploration early in training with exploitation later.

8 Acknowledgments

We would like to thank Amber Xie, Joey Hejna, Suvir Mirchandani, Hung-Chieh Fang and Andy Tang for feedback on an earlier version of the paper. We thank Ayush Chakravarthy for helpful comments on the experimental evaluations. We thank Yoonho Lee for several valuable comments on formalizing diversity. We thank Hamidur Rahman and Pervin Akhter for helpful guidance on the formalization of the problem and feedback on the project in general. We thank Zabed Hasan and Fazilat Afreen for their insightful feedback on experimental evaluations. This work is supported by an NSF CAREER award, NSF award # 1941722, ONR YIP, DARPA YFA award # W911NF2210214, and Schmidt Sciences.

References

Azar, Mohammad Gheshlaghi, Ian Osband, and Rémi Munos (2017). *Minimax Regret Bounds for Reinforcement Learning*. arXiv: 1703.05449 [stat.ML]. URL: https://arxiv.org/abs/1703.05449.

Bhatnagar, Shalabh et al. (2009). "Natural actor–critic algorithms". In: *Automatica* 45.11, pp. 2471–2482. ISSN: 0005-1098. DOI: https://doi.org/10.1016/j.automatica.2009.07.008. URL: https://www.sciencedirect.com/science/article/pii/S0005109809003549.

Chevalier-Boisvert, Maxime et al. (2019). *BabyAI: A Platform to Study the Sample Efficiency of Grounded Language Learning*. arXiv: 1810.08272 [cs.AI]. URL: https://arxiv.org/abs/1810.08272.

Chow, Yinlam et al. (2024). *Inference-Aware Fine-Tuning for Best-of-N Sampling in Large Language Models*. arXiv: 2412.15287 [cs.CL]. URL: https://arxiv.org/abs/2412.15287.

Cui, Ganqu et al. (2025). *The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models*. arXiv: 2505.22617 [cs.LG]. URL: https://arxiv.org/abs/2505.22617.

DeepSeek-AI et al. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv: 2501.12948 [cs.CL]. URL: https://arxiv.org/abs/2501.12948.

Degris, Thomas, Martha White, and Richard S. Sutton (2013). *Off-Policy Actor-Critic*. arXiv: 1205.4839 [cs.LG]. URL: https://arxiv.org/abs/1205.4839.

Friedman, Dan and Adji Bousso Dieng (2023). *The Vendi Score: A Diversity Evaluation Metric for Machine Learning*. arXiv: 2210.02410 [cs.LG]. URL: https://arxiv.org/abs/2210.02410.

Haarnoja, Tuomas, Haoran Tang, et al. (2017). *Reinforcement Learning with Deep Energy-Based Policies*. arXiv: 1702.08165 [cs.LG]. URL: https://arxiv.org/abs/1702.08165.

Haarnoja, Tuomas, Aurick Zhou, et al. (2018). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. arXiv: 1801.01290 [cs.LG]. URL: https://arxiv.org/abs/1801.01290.

He, Andre, Daniel Fried, and Sean Welleck (2025). *Rewarding the Unlikely: Lifting GRPO Beyond Distribution Sharpening*. arXiv: 2506.02355 [cs.LG]. URL: https://arxiv.org/abs/2506.02355.

Islam, Riashat, Zafarali Ahmed, and Doina Precup (2019). *Marginalized State Distribution Entropy Regularization in Policy Optimization*. arXiv: 1912.05128 [cs.LG]. URL: https://arxiv.org/abs/1912.05128.

Kakade, Sham M (2001). "A Natural Policy Gradient". In: *Advances in Neural Information Processing Systems*. Ed. by T. Dietterich, S. Becker, and Z. Ghahramani. Vol. 14. MIT Press. URL: https://proceedings.neurips.cc/paper_files/paper/2001/file/4b86abe48d358ecf194c56c69108433e-Paper.pdf.

Kakade, Sham M. and John Langford (2002). "Approximately Optimal Approximate Reinforcement Learning". In: *International Conference on Machine Learning*. URL: https://api.semanticscholar.org/CorpusID:31442909.

Kazemnejad, Amirhossein et al. (2025). *VinePPO: Refining Credit Assignment in RL Training of LLMs*. arXiv: 2410.01679 [cs.LG]. URL: https://arxiv.org/abs/2410.01679.

Kumar, Saurabh et al. (2020). *One Solution is Not All You Need: Few-Shot Extrapolation via Structured MaxEnt RL*. arXiv: 2010.14484 [cs.LG]. URL: https://arxiv.org/abs/2010.14484.

Lanchantin, Jack et al. (2025). *Diverse Preference Optimization*. arXiv: 2501.18101 [cs.CL]. URL: https://arxiv.org/abs/2501.18101.

Li, Tianjian et al. (2025). *Jointly Reinforcing Diversity and Quality in Language Model Generations*. arXiv: 2509.02534 [cs.CL]. URL: https://arxiv.org/abs/2509.02534.

Lillicrap, Timothy P. et al. (2016). "Continuous control with deep reinforcement learning". In: *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*. Ed. by Yoshua Bengio and Yann LeCun. URL: http://arxiv.org/abs/1509.02971.

Liu, Zichen et al. (2025). *Understanding R1-Zero-Like Training: A Critical Perspective*. arXiv: 2503.20783 [cs.LG]. URL: https://arxiv.org/abs/2503.20783.

Nagarajan, Vaishnavh et al. (2025). *Roll the dice and look before you leap: Going beyond the creative limits of next-token prediction*. arXiv: 2504.15266 [cs.LG]. URL: https://arxiv.org/abs/2504.15266.

OpenAI et al. (2024). *OpenAI o1 System Card*. arXiv: 2412.16720 [cs.AI]. URL: https://arxiv.org/abs/2412.16720.

Ouyang, Long et al. (2022). *Training language models to follow instructions with human feedback*. arXiv: 2203.02155 [cs.CL]. URL: https://arxiv.org/abs/2203.02155.

Pathak, Deepak et al. (2017). "Curiosity-driven Exploration by Self-supervised Prediction". In: *Proceedings of the 34th International Conference on Machine Learning*. Ed. by Doina Precup and Yee Whye Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, pp. 2778–2787. URL: https://proceedings.mlr.press/v70/pathak17a.html.

Qwen (2025). *QwQ-32B: Embracing the Power of Reinforcement Learning*. URL: https://qwenlm.github.io/blog/qwq-32b/.

Roux, Nicolas Le et al. (2025). *Tapered Off-Policy REINFORCE: Stable and efficient reinforcement learning for LLMs*. arXiv: 2503.14286 [cs.LG]. URL: https://arxiv.org/abs/2503.14286.

Schulman, John, Sergey Levine, et al. (2017). *Trust Region Policy Optimization*. arXiv: 1502.05477 [cs.LG]. URL: https://arxiv.org/abs/1502.05477.

Schulman, John, Philipp Moritz, et al. (2018). *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. arXiv: 1506.02438 [cs.LG]. URL: https://arxiv.org/abs/1506.02438.

Schulman, John, Filip Wolski, et al. (2017). *Proximal Policy Optimization Algorithms*. arXiv: 1707.06347 [cs.LG]. URL: https://arxiv.org/abs/1707.06347.

Seo, Younggyo et al. (2021). *State Entropy Maximization with Random Encoders for Efficient Exploration*. arXiv: 2102.09430 [cs.LG]. URL: https://arxiv.org/abs/2102.09430.

Snell, Charlie et al. (2024). *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*. arXiv: 2408.03314 [cs.LG]. URL: https://arxiv.org/abs/2408.03314.

Song, Yuda, Julia Kempe, and Remi Munos (2025). *Outcome-based Exploration for LLM Reasoning*. arXiv: 2509.06941 [cs.LG]. URL: https://arxiv.org/abs/2509.06941.

Sutton, Richard S. and Andrew G. Barto (2018). *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book. ISBN: 0262039249.

Sutton, Richard S., David McAllester, et al. (1999). "Policy gradient methods for reinforcement learning with function approximation". In: *Proceedings of the 13th International Conference on Neural Information Processing Systems*. NIPS'99. Denver, CO: MIT Press, pp. 1057–1063.

Tajwar, Fahim, Yiding Jiang, et al. (2025). *Training a Generally Curious Agent*. arXiv: 2502.17543 [cs.LG]. URL: https://arxiv.org/abs/2502.17543.

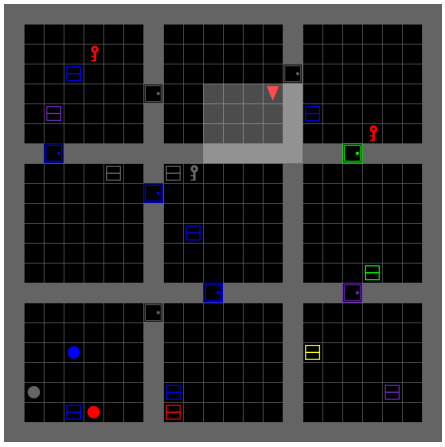
Tajwar, Fahim, Guanning Zeng, et al. (2026). *Maximum Likelihood Reinforcement Learning*. arXiv: 2602.02710 [cs.LG]. URL: https://arxiv.org/abs/2602.02710.
Tang, Yunhao et al. (2025). *Optimizing Language Models for Inference Time Objectives using Reinforcement Learning*. arXiv: 2503.19595 [cs.LG]. URL: https://arxiv.org/abs/2503.19595.
Wang, Ziyu et al. (2017). *Sample Efficient Actor-Critic with Experience Replay*. arXiv: 1611.01224 [cs.LG]. URL: https://arxiv.org/abs/1611.01224.
Williams, Ronald J. (May 1992). "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning". In: *Mach. Learn.* 8.3–4, pp. 229–256. ISSN: 0885-6125. DOI: 10.1007/BF00992696. URL: https://doi.org/10.1007/BF00992696.
Wu, Fang et al. (2025). *The Invisible Leash: Why RLVR May Not Escape Its Origin*. arXiv: 2507.14843 [cs.LG]. URL: https://arxiv.org/abs/2507.14843.
Younis, Omar G. et al. (Sept. 2024). *Minari*. Version 0.5.0. DOI: 10.5281/zenodo.13767625. URL: https://github.com/Farama-Foundation/Minari.
Yu, Qiying et al. (2025). *DAPO: An Open-Source LLM Reinforcement Learning System at Scale*. arXiv: 2503.14476 [cs.LG]. URL: https://arxiv.org/abs/2503.14476.
Yue, Yang et al. (2025). *Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?* arXiv: 2504.13837 [cs.AI]. URL: https://arxiv.org/abs/2504.13837.
Zhang, Yiming et al. (2025). *NoveltyBench: Evaluating Language Models for Humanlike Diversity*. arXiv: 2504.05228 [cs.CL]. URL: https://arxiv.org/abs/2504.05228.
Zhao, Rosie et al. (2025). *Echo Chamber: RL Post-training Amplifies Behaviors Learned in Pretraining*. arXiv: 2504.07912 [cs.LG]. URL: https://arxiv.org/abs/2504.07912.
Zheng, Chujie et al. (2025). *Group Sequence Policy Optimization*. arXiv: 2507.18071 [cs.LG]. URL: https://arxiv.org/abs/2507.18071.

A Implementation Details

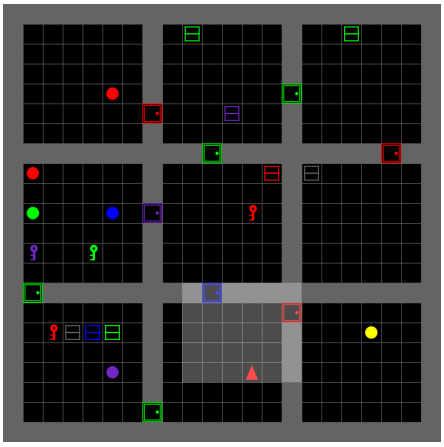
BabyAI and MiniGrid. For BabyAI tasks, the policy conditions on the grid image, the agent's direction embedding, and the mission text (encoded by a GRU); the action space is the standard BabyAI/MiniGrid discrete set *left, right, forward, pickup, drop, toggle, done*. We provide example configurations for each task in Fig. 5. We train a CNN–GRU policy that outputs action logits. In *MiniGrid–FourRooms*, the action space is identical, but the observation excludes a mission (as the goal specification is fixed), so we use a compact MLP that produces action logits conditioned on a flattened image observation. During pretraining, we use an 80/20 train–test split of expert demonstrations for each task, except for *Synthseq* and *BossLevel*, where we found that the full dataset was required to obtain a reasonably strong base policy. We used the dataset from Younis et al. (2024). We pretrain by minimizing the cross-entropy loss with an entropy regularizer.

During RLFT, we fine-tune on 50 configurations. For all tasks except *BossLevel* and *Synthseq*, these configurations are drawn from the pretraining test set; for *BossLevel* and *Synthseq*, we did not carve out a separate test set. If the agent reaches the goal at time t (episode horizon $H{=}100$), it receives the reward $1 - 0.5 \cdot \frac{t}{H}$; otherwise $r = 0$. (BabyAI/MiniGrid defaults to $1 - 0.9 \cdot \frac{t}{H}$; we found that lowering the time penalty improved diversity during RLFT, especially for REINFORCE, by reducing the disadvantage of longer but successful trajectories.)

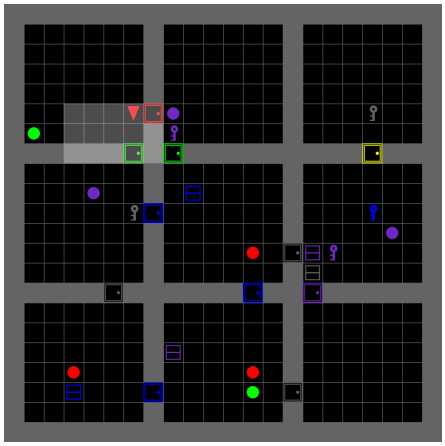
Algorithmic Creativity (Triangle Discovery). In *Triangle Discovery*, an agent must output a sequence of edge tokens that form a valid triangle in an *unobserved* undirected graph. The input sequence comprises a graph index plus a prefix prompt and the agent's past outputs. We set the maximum sequence length to be 11. The action space is a discrete vocabulary of size 1017. We pretrain a decoder-only Transformer with masked cross-entropy. The pretraining dataset, from Nagarajan et al. (2025), contains 15,000 samples per graph, with the same being a triangle with probability $\frac{1}{3}$ and an edge with probability $\frac{2}{3}$ edges. Each graph has 999 nodes. For RLFT, we fine-tune on 3 fixed graphs. The reward is sparse: $+1$ for a valid triangle, 0 otherwise.



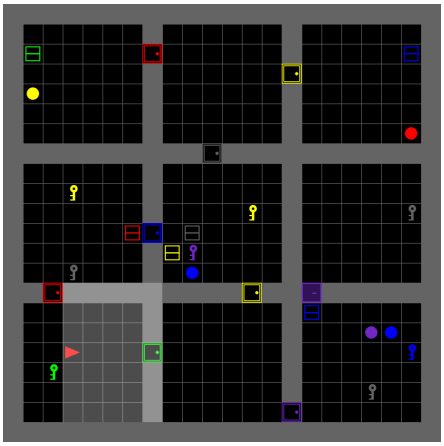
(a) *GoTo:* mission "go to purple box."



(b) *Pickup:* mission "pick up a green ball."



(c) *Synthseq:* mission "put the ball on your right next to a red ball and pick up a purple ball after you open a grey door."



(d) *Bosslevel:* mission "put a yellow key next to the blue key and pick up a ball after you pick up a yellow key."

Figure 5: Example BabyAI environments and their missions.

A.1 Polychromic PPO

We summarize implementation details for polychromic PPO , beginning with the *vine sampling* scheme used for on-policy data collection, then additional stability techniques and full pseudocode in Algorithm 2.

A.1.1 Vine Sampling

In this section, we describe the vine sampling scheme (Schulman, Levine, et al., 2017) we used for polychromic PPO . In vine sampling, we first generate a number of trajectories. Then, we select a subset of the states visited, denoted by $\{s_1, \ldots, s_p\}$ - this is called the *rollout set*. For each state s_i in the rollout set (we call this a *rollout state*), we generate a set of trajectories $\tau_{1:N} \sim \pi_\beta(\cdot \mid s_i)$. To avoid notational overload and to make sample accounting explicit, we distinguish:

- n: the set size used by the set-RL objective (number of trajectories per set)
- N: the number of *rollouts* collected from each rollout state/vine state
- p: the number of *rollout states* selected along each seed rollout

- B: the trajectory budget i.e., maximum number of trajectories we can collect.

We now discuss the method we used to select the set of rollout states. Our goal is to identify multiple states from which we want to generate vines so that we can use the polychromic advantage to update the policy at a large number of states. Suppose, we have a trajectory budget B, i.e., we are allowed to generate at most B trajectories during the on-policy data collection. We first sample N rollouts independently where $N > n$. Now, for each of these N trajectories, we identify p rollout states according to some criterion. Some examples of rollout state criterion are: (1) Top p states with the highest entropy; sample more trajectories from states where the policy is more uncertain, (2) Top p states with the highest critic losses; sample more trajectories where our critic is wrong/biased, and (3) p equally spaced out states. Suppose the main trajectory is T timesteps long. We select the states at timestep $\frac{T}{p+1}, \frac{2T}{p+1}, \cdots, \frac{pT}{p+1}$.

In this paper, we used the third criterion. Note that, in this scheme, we generate, in total, $N + N^2(p-1)$ trajectories. Since we are allowed to sample at most B trajectories, we must select N and p such that $N > n$ and $N + N^2(p-1) \leq B$. Across all environments, we set a trajectory budget $B = 136$ for all methods. As such, we use sets of size $n = 4$ for set RL, $N = 8$ vines at each rollout state, and $p = 2$ rollout states per trajectory. Note that at each of these rollout states, given we generate N rollouts, we can find $\binom{N}{n}$ sets for our set RL algorithm. In fact, this combinatorial construction of sets allows us to compute a baseline that is unbiased. In our experiments, we set the number of sets, M, to be 4. For our chosen hyperparameters, this was sufficiently large number of sets from each rollout state allowing us to use several sets to compute the baseline.

A.1.2 Other Implementation Details and Pseudocode

Now, we will discuss other implementation details for polychromic PPO . We provide the pseudocode for the complete algorithm in Algorithm 2 and the hyperparameters in Table 3.

We found that adding the KL penalty from the behavior policy was helpful for stability. In the absence of it, in some tasks, the model's performance collapses after a certain number of training epochs. This is likely because our method explicitly encourages exploration and the KL penalty provides an anchor that prevents the model from drifting too far.

In practical implementation, we found that adding a window within which we update all the advantages to the polychromic advantage to be useful. As shown in Algorithm 2, we do not only set the advantage at the rollout state s_t to be the polychromic advantage - instead, we set the advantages at states $s_t, s_{t+1}, \cdots, s_{t+W}$ to be the polychromic advantage even though we do not generate vines from $s_{t+1}, \cdots, s_{t+W}$. This ensures that the exploratory behavior that the polychromic advantage encourages is induced at all states throughout the window. Otherwise, although the policy might be exploratory at s_t, it might revert to being purely exploitative at all subsequent states due to the updates using the standard PPO objective; this would cause the policy to not explore. This issue becomes pronounced in environments where, despite being exploratory at s_t, the exploitative behavior in all subsequent states may override the exploration in s_t. This is why this implementation trick was important in BabyAI and Minigrid while, in Algorithmic Creativity, we set $W = 0$ since once the policy visits a diverse set of nodes from s_t, it is not easy to merge paths.

Algorithm 2 Polychromic PPO

Require: pretrained Policy π_β, value function V_ϕ, discount γ, GAE parameter λ, clipping ϵ, policy epochs K, value coef c_v, KL target coef β_{KL}
(Poly-PPO specific:) number of sets N, set size n, number of vine states p, number of vines N, window length W.

Initialize $\pi_\theta \leftarrow \pi_\beta$
for iteration $= 1, 2, \ldots$ **do**
  Collect on-policy data using vine sampling:
    Initialize rollout_states $\leftarrow \{\}$ ▷ dictionary: state $\mapsto$ list of trajectories
    Roll out N trajectories using π_β
    for each trajectory τ **do**
      select p vine states from τ
      for each vine state s **do**
        Roll out N trajectories from s using π_β
        Append the new trajectories to rollout_states$[s]$
      end for
    end for
  Compute advantages:
  if $s \notin$ rollout_states **then**
    $\delta_t \leftarrow r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
    $A_t \leftarrow \mathrm{GAE}(\delta_{t:T}, \gamma, \lambda)$ ▷ generalized advantage estimation
    $\hat{R}_t \leftarrow A_t + V_\phi(s_t)$
  else if $s \in$ rollout_states **then**
    Create groups $g_1, g_1, \cdots, g_M$ of n trajectories from rollout_states$[s]$.
    Compute set scores $\mathrm{score}(g_i) = f_{\mathrm{poly}}(s, \tau_{1:n})$ for $\tau_{1:n} \in g_i$(Eq. (7))
    Compute baseline $\hat{f}(s) = \frac{1}{M}\sum_{i=1}^{M} \mathrm{score}(g_i)$.
    Define polychromic advantage of pairs $(s_t, a_t), \ldots, (s_{t+W}, a_{t+W}) \in \tau$ for each $\tau \in g_i$:

$$A^{\mathrm{poly}}(s_{t'}, a_{t'}) \leftarrow \mathrm{score}(g_i) - \hat{f}(s) \qquad \triangleright \text{ polychromic advantage}$$

  end if
  Normalize $\{A_t\}$
  for epoch $= 1, \ldots, K$ **do**
    for minibatch $\mathcal{B}$ **do**
      Compute ratios $r_t(\theta) \leftarrow \frac{\pi_\theta(a_t|s_t)}{\pi_\beta(a_t|s_t)}$
      Policy loss:

$$\mathcal{L}_\pi(\theta) \;=\; -\frac{1}{|\mathcal{B}|}\sum_{t\in\mathcal{B}} \min\Big(r_t(\theta)A_t,\; \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\, A_t\Big)$$

      Value loss: $\mathcal{L}_V(\phi) = \frac{1}{|\mathcal{B}|}\sum_{t\in\mathcal{B}} \big(V_\phi(s_t) - \hat{R}_t\big)^2$
      KL penalty: $\mathcal{L}_{\mathrm{KL}}(\theta) = \frac{1}{|\mathcal{B}|}\sum_{t\in\mathcal{B}} \mathrm{KL}\big(\pi_\beta(\cdot|s_t) \,\|\, \pi_\theta(\cdot|s_t)\big)$
      Total loss:

$$\mathcal{L}(\theta, \phi) \;=\; \mathcal{L}_\pi(\theta) \;+\; c_v\, \mathcal{L}_V(\phi) \;+\; \beta_{\mathrm{KL}}\, \mathcal{L}_{\mathrm{KL}}(\theta)$$

      Take gradient step on θ, ϕ to minimize $\mathcal{L}$
      Update $\pi_\beta \leftarrow \pi_\theta$
    end for
  end for
end for

Hyperparameter	Value
PPO epochs	2
Minibatch size	64
Discount (γ)	1.0
GAE parameter (λ)	0.95
Clipping parameter (ϵ)	0.2
Actor learning rate	1×10^{-5}
Critic learning rate	1×10^{-4}
Value loss coefficient (c_v)	0.5
KL coefficient (β_{KL})	{0.005, 0.01, 0.05, 0.1}
Max grad norm	0.5
Temperature	1.0
Num. vines at state (N)	8
Size of sets (n)	4
Num. sets (M)	4
Num. rollout states (p)	2
Polychrome window (W)	5 for BabyAI/Minigrid, 0 for Algorithmic Creativity

Table 3: Polychromic PPO hyperparameters. All hyperparameters are fixed apart from the KL coefficient β_{KL} for which we provide the set we sweep over.

B Generalization Experiment

To evaluate the agent's ability to generalize to initial state perturbations, we designed the following experiment. First, for each environment seed, we perform 100 rollouts with the pretrained policy to identify all reachable rooms. Then, for each unique room visited, we evaluate the target policy (fine-tuned using RL) by placing the agent at 10 randomly selected starting positions within that room. Performance is measured using the pass@1 metric, which calculates the success rate on the first attempt from these novel starting points. This randomization presents a significant challenge, as a successful trajectory from a new initial state often requires substantially different strategies than those effective from the original start state as shown in Fig. 6.

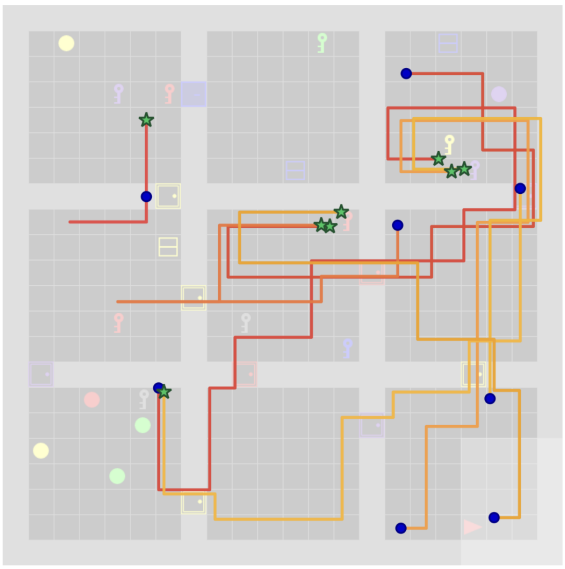


Figure 6: Generalization under state perturbations in BabyAI BossLevel environment. The mission here is "*Pickup a key*". The figure shows successful rollouts across perturbed initial states (blue circles), highlighting diverse strategies learned by the agent.

C Proofs

C.1 Proof of Lemma 3.2

Recall our setup where, from each state s visited during on-policy rollouts, we sample n actions, $a_{1:n}$. As such, from each state, we generate a tree where each node (corresponding to a state) branches out into n children. We denote all the nodes at depth t of this tree by $s_t^{(1)}, \cdots, s_t^{(n^t)}$. Furthermore, if, from a state s, the agent samples the set of actions $a_{1:n}$, the agent gets a reward $f(s, a_{1:n})$ where f is the our objective function to be used in set RL. We will assume that f is normalized between 0 and 1. For simplicity, we assume that the initial state is fixed. Note that we can construct the distribution of states visited by policy π_θ at time t in this tree as follows: for $j \in \{1, \cdots, n^t\}$, the probability of reaching a state at the t-th timestep and in the j-th node at that level is:

$$
\begin{aligned}
&\mathbb{P}(s_0 \to s_t^{(j)} = s, t, \pi_\theta) \\
&= \sum_{(a_0)_{1:n}} \pi_\theta((a_0)_{1:n} \mid s_0) \sum_{s_1^{(1:n)}} P(s_1^{(1:n)}) \mid s_0, (a_0)_{1:n}) \cdots \\
&\times \sum_{(a_{t-1})_{1:n}^{(1)}, \cdots, (a_{t-1})_{1:n}^{(n^{t-1})}} \pi_\theta((a_{t-1})_{1:n}^{(1)}, \cdots, (a_{t-1})_{1:n}^{(n^{t-1})} \mid s_{t-1}^{(1)}, \cdots, s_{t-1}^{(n^{t-1})}) \\
&\times P(s_t^{(j)} \mid s_{t-1}^{(1:n^{t-1})}, (a_{t-1})_{1:n}^{(1)}, \cdots, (a_{t-1})_{1:n}^{(n^{t-1})})
\end{aligned}
$$

Here, each $\pi_\theta(a_{1:n} \mid s) = \prod_{i=1}^n \pi_\theta(a_i \mid s)$. Similarly, we use the following shorthand:

$$
\pi_\theta((a_{t-1})_{1:n}^{(1)}, \cdots, (a_{t-1})_{1:n}^{(n^{t-1})} \mid s_{t-1}^{(1)}, \cdots, s_{t-1}^{(n^{t-1})}) = \prod_{i=1}^{n^{t-1}} \pi_\theta((a_{t-1})_{1:n}^{(i)} \mid s_{t-1}^{(i)}).
$$

Before we prove the lemma, we make the following observation: suppose the environment's state transition dynamics is deterministic (i.e., the distribution $P(s_{t+1} \mid s_t, a_t)$ is a Dirac delta distribution), then the set Q-function can be written using the set value function as follows:

$$Q_\pi^\sharp(s, a_{1:n}) = f(s, a_{1:n}) + \gamma \sum_{i=1}^{n} V_\pi^\sharp(s_1^{(i)})$$

where $s_1^{(1)}, \cdots, s_1^{(n)}$ are the states reached from s after taking actions $a_{1:n}$ independently. One can verify this by using the definition set Q-functoin and noting that:

$$\begin{aligned}
&\mathbb{E}\left[\sum_{t=1}^{\infty}\gamma^t \sum_{i=1}^{n^t} f(s_t^{(i)}, (a_t)_{1:n}^{(i)}) \mid s_0 = s, a_{1:n}(s_0) = a_{1:n}\right] \\
&= \mathbb{E}\left[\gamma \sum_{i=1}^{n} f(s_1^{(i)}, (a_1)_{1:n}^{(i)} + \sum_{t=2}^{\infty}\gamma^t \sum_{i=1}^{n^t} f(s_t^{(i)}, (a_t)_{1:n}^{(i)}) \Big| s_0 = s, (a_0)_{1:n} = a_{1:n}\right] \\
&= \mathbb{E}_{(a_1)_{1:n}^{(i)}}\left[\gamma \sum_{i=1}^{n} f(s_1^{(i)}, (a_1)_{1:n}^{(i)}) + \mathbb{E}\left[\sum_{t=2}^{\infty}\gamma^t \sum_{i=1}^{n^t} f(s_t^{(i)}, (a_t)_{1:n}^{(i)}) \Big|_{s_1^{(i)}, (a_1)_{1:n}^{(i)}}^{s_0=s, (a_0)_{1:n}=a_{1:n}}\right] \Big| \cdots\right] \\
&= \gamma \sum_{i=1}^{n} \mathbb{E}_{(a_1)_{1:n}^{(i)}}\left[f(s_1^{(i)}, (a_1)_{1:n}^{(i)}) + \mathbb{E}\left[\sum_{t=2}^{\infty}\gamma^{t-1} \sum_{j=1}^{n^{t-1}} f(s_t^{(n(i-1)+j)}, (a_t)_{1:n}^{(n(i-1)+j)}) \Big| \cdots\right] \Big| \cdots\right] \\
&= \gamma \sum_{i=1}^{n} V_\pi^\sharp(s_1^{(i)})
\end{aligned}$$

Now we prove Lemma 3.2. Our proof follows the same procedure as in the proof of the standard performance difference lemma in Sham M. Kakade et al. (2002). We first prove the following lemma:

Lemma C.1 *Given the state visitation tree generated by policy π_θ and any normalized objective function f,*

$$\mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t \sum_{i=1}^{n^t} f(s_t^{(i)}, (a_t)_{1:n}^{(i)})\right] = \frac{1}{1-\gamma n}\mathbb{E}_{s\sim d_{\pi_\theta}^\sharp(s), a_{1:n}\sim\pi_\theta(\cdot|s)}\left[f(s, a_{1:n})\right] \tag{8}$$

where

$$d_\pi^\sharp(s) = (1-\gamma n)\sum_{t=0}^{\infty}\sum_{i=1}^{n}\gamma^t \mathbb{P}(s_0 \to s_t^{(i)}, t, \pi_\theta).$$

Proof.

$$\begin{aligned}
&\mathbb{E}\left[\sum_{t=0}^{\infty}\sum_{i=1}^{n^t}\gamma^t f(s_t^{(i)}, (a_t)_{1:n}^{(i)})\right] \\
&= \sum_{t=0}^{\infty}\gamma^t \sum_{i=1}^{n^t}\mathbb{E}\left[f(s_t^{(i)}, (a_t)_{1:n}^{(i)})\right] \\
&= \sum_{t=0}^{\infty}\gamma^t \sum_{i=1}^{n^t}\sum_{s_t^{(i)}}\mathbb{P}(s_0 \to s_t^{(i)}, t, \pi_\theta)\mathbb{E}_{a_{1:n}(s_t^{(i)})}\left[f(s_t^{(i)}, (a_t)_{1:n}^{(i)})\right] \\
&= \frac{1}{1-\gamma n}(1-\gamma n)\sum_{t=0}^{\infty}\gamma^t \sum_{i=1}^{n^t}\sum_{s_t^{(i)}}\mathbb{P}(s_0 \to s_t^{(i)}, t, \pi_\theta)\mathbb{E}_{a_{1:n}(s_{t,i})}\left[f(s_t^{(i)}, (a_t)_{1:n}^{(i)})\right] \\
&= \frac{1}{1-\gamma n}\mathbb{E}_{s\sim d_{\pi_\theta}^\sharp(s), a_{1:n}\sim\pi_\theta(\cdot|s)}\left[f(s, a_{1:n}\right].
\end{aligned}$$

□

Lemma 3.2 (restated). Given any two policies π_θ and π_β,

$$V^\sharp_{\pi_\theta}(s) - V^\sharp_{\pi_\beta}(s) = \frac{1}{1-\gamma n}\mathbb{E}_{s\sim d^\sharp_{\pi_\theta}(\cdot), a_{1:n}\sim\pi_\theta(\cdot|s)}\left[A^\sharp_{\pi_\beta}(s, a_{1:n})\right]. \tag{9}$$

Proof. We first, show that

$$V^\sharp_{\pi_\theta}(s) - V^\sharp_{\pi_\beta}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty}\sum_{i=1}^{n^t} Q^\sharp_{\pi_\beta}(s_t^{(i)}, (a_t)^{(i)}_{1:n}) - V^\sharp_{\pi_\beta}(s_t^{(i)}) \mid s_0 = s\right]. \tag{10}$$

We show this by the following simplification:

$$\begin{aligned}
&V^\sharp_{\pi_\theta}(s) - V^\sharp_{\pi_\beta}(s)\\
&= \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{n^t} f\big(s_t^{(i)}, (a_t)^{(i)}_{1:n}\big) \,\middle|\, s_0 = s\right] - V^\sharp_{\pi_\beta}(s)\\
&= \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{n^t} f\big(s_t^{(i)}, (a_t)^{(i)}_{1:n}\big) \,\middle|\, s_0 = s\right] - V^\sharp_{\pi_\beta}(s)\\
&\quad + \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{n^t}\gamma\sum_{j=1}^{n} V^\sharp_{\pi_\beta}(s_{t+1}^{(n(i-1)+j)}) \,\middle|\, s_0 = s\right] - \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{n^t}\gamma\sum_{j=1}^{n} V^\sharp_{\pi_\beta}(s_{t+1}^{(n(i-1)+j)}) \,\middle|\, s_0 = s\right]\\
&= \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{n^t}\Big(f\big(s_t^{(i)}, (a_t)^{(i)}_{1:n}\big) + \gamma\sum_{j=1}^{n} V^\sharp_{\pi_\beta}(s_{t+1}^{(n(i-1)+j)})\Big)\right] - \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{n^t} V^\sharp_{\pi_\beta}(s_t^{(i)}) \,\middle|\, s_0^{(1)} = s\right].
\end{aligned}$$

Now, the first term on the right hand side can be simplified (we suppress the condition that $s_0 = s$ in terms of notation):

$$\begin{aligned}
&\mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{n^t}\Big(f\big(s_t^{(i)}, (a_t)^{(i)}_{1:n}\big) + \gamma\sum_{j=1}^{n} V^\sharp_{\pi_\beta}(s_{t+1}^{(n(i-1)+j)})\Big)\right]\\
&= \sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{n^t}\mathbb{E}_{\pi_\theta}\left[f\big(s_t^{(i)}, (a_t)^{(i)}_{1:n}\big) + \gamma\sum_{j=1}^{n} V^\sharp_{\pi_\beta}(s_{t+1}^{(n(i-1)+j)})\right]\\
&= \sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{n^t}\mathbb{E}_{s_t^{(i)}, (a_t)^{(i)}_{1:n}}\left[\mathbb{E}\left[f\big(s_t^{(i)}, (a_t)^{(i)}_{1:n}\big) + \gamma\sum_{j=1}^{n} V^\sharp_{\pi_\beta}(s_{t+1}^{(n(i-1)+j)})\Big| s_t^{(i)}, (a_t)^{(i)}_{1:n}\right]\right]\\
&= \sum_{t=0}^{\infty}\sum_{i=1}^{n^t}\mathbb{E}_{s_t^{(i)}, (a_t)^{(i)}_{1:n}}\left[Q^\sharp_{\pi_\beta}(s_t^{(i)}, (a_t)^{(i)}_{1:n})\right]\\
&= \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{n^t} Q^\sharp_{\pi_\beta}(s_t^{(i)}, (a_t)^{(i)}_{1:n})\right]
\end{aligned}$$

Therefore,

$$V^{\sharp}_{\pi_\theta}(s) - V^{\sharp}_{\pi_\beta}(s)$$
$$= \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t \sum_{i=1}^{n^t} Q^{\sharp}_{\pi_\beta}(s_t^{(i)}, (a_t)^{(i)}_{1:n}) - V^{\sharp}_{\pi_\beta}(s_t^{(i)}) \middle| s_0 = s\right].$$

Then, using Eq. (8), the statement follows.

□

C.2 Proof of Proposition 5.1

We follow the set-up in Cui et al. (2025). We parameterize the policy as a softmax distribution:

$$\pi_\theta(a \mid s) = \frac{\exp(z_{sa})}{\sum_{a'}\exp(z_{sa'})}$$

where z_{sa} is the output logit of action a from state s. We also have the following derivative:

$$\frac{\partial}{\partial z_{sa'}}\log \pi_\theta(a \mid s) = 1\{a' = a\} - \pi_\theta(a' \mid s).$$

First, we prove the following that shows how the output logit changes after one-step parameter update in the set RL paradigm.

Lemma C.2 *Given the softmax policy π_θ is updated using Eq. (4) using a step-size α, the change in the output logit z_{sa} after one step update is given by*

$$z^{k+1}_{sa} - z^k_{sa} = \alpha \mathbb{E}_{a_{1:n}\sim\pi^k_\theta(\cdot|s)}\left[f(s, a_{1:n})\sum_{i=1}^{n} 1\{a_i = a\}\right] - \alpha n \pi^k_\theta(a \mid s)\mathbb{E}_{a_{1:n}\sim\pi^k_\theta(\cdot|s)}\left[f(s, a_{1:n})\right].$$

Proof. This can be proven by elementary properties of expectation:

$$\begin{aligned}
z^{k+1}_{sa} - z^k_{sa} &= \alpha \frac{\partial}{\partial z^k_{sa}}\mathbb{E}_{a_{1:n}\sim\pi^k_\theta(\cdot|s)}\left[f(s, a_{1:n})\right] \\
&= \alpha \mathbb{E}_{a_{1:n}\sim\pi^k_\theta(\cdot|s)}\left[\frac{\partial}{\partial z^k_{sa}}\log(\mathbb{P}(a_{1:n} \mid s))f(s, a_{1:n})\right] \\
&= \alpha \mathbb{E}_{a_{1:n}\sim\pi^k_\theta(\cdot|s)}\left[\sum_{i=1}^{n}\frac{\partial}{\partial z^k_{sa}}\log(\pi^k_\theta(a_i \mid s))f(s, a_{1:n})\right] \\
&= \alpha \mathbb{E}_{a_{1:n}\sim\pi^k_\theta(\cdot|s)}\left[\sum_{i=1}^{n}\left(1\{a_i = a\} - \pi^k_\theta(a \mid s)\right)f(s, a_{1:n})\right]
\end{aligned}$$

□

Now we prove the main result:

Proposition 5.1 (restated). Consider the set-RL setup at state s. After one update to the policy, the change in entropy, $\Delta = \mathcal{H}\left(\pi^{k+1}_\theta \mid s\right) - \mathcal{H}\left(\pi^k_\theta \mid s\right)$, is given by

$$\Delta \approx -\alpha \mathrm{Cov}_{a_{1:n}}\left(\frac{1}{n}\sum_{i=1}^{n}\log \pi^k_\theta(a_i \mid s), \mathrm{Cov}_{a'_{1:n}}\left(f(s, a'_{1:n}), \sum_{i,j=1}^{n} 1\{a_i = a'_j\}\right)\right),$$

where both covariances are taken with respect to $\pi_\theta^k(\cdot \mid s)$.

Proof. We have the following first order approximation of Δ (Cui et al., 2025):

$$\Delta \approx -\mathrm{Cov}_{a\sim\pi_\theta^k(\cdot|s)}\left(\log \pi_\theta^k(a \mid s), z_{sa}^{k+1} - z_{sa}^k\right).$$

Using Lemma C.2, we get that

$$\begin{aligned}\Delta \approx & \alpha n \mathbb{E}[f(s,a_{1:n})]\mathrm{Cov}_{a\sim\pi_\theta^k(\cdot|s)}\left(\log \pi_\theta^k(a \mid s), \pi_\theta^k(a \mid s)\right) \\ & - \alpha \mathrm{Cov}_{a\sim\pi_\theta^k(\cdot|s)}\left(\log \pi_\theta^k(a \mid s), \mathbb{E}_{a_{1:n}\sim\pi_\theta^k(\cdot|s)}\left[f(s,a_{1:n})\sum_{i=1}^n 1\{a_i = a\}\right]\right)\end{aligned}$$

Note that

$$\begin{aligned}\mathrm{Cov}_{a_{1:n}}\left(f(s,a_{1:n}), \sum_{i=1}^n 1\{a_i = a\}\right) = & \mathbb{E}_{a_{1:n}\sim\pi_\theta^k(\cdot|s)}\left[f(s,a_{1:n})\sum_{i=1}^n 1\{a_i = a\}\right] \\ & - n\pi_\theta^k(a \mid s)\mathbb{E}_{a_{1:n}\sim\pi_\theta^k(\cdot|s)}\left[f(s,a_{1:n})\right]\end{aligned}$$

Using this, we get that

$$\begin{aligned}\Delta \approx & \alpha n \mathbb{E}[f(s,a_{1:n})]\mathrm{Cov}_{a\sim\pi_\theta^k(\cdot|s)}\left(\log \pi_\theta^k(a \mid s), \pi_\theta^k(a \mid s)\right) \\ & - \alpha \mathrm{Cov}_{a\sim\pi_\theta^k(\cdot|s)}\left(\log \pi_\theta^k(a \mid s), \mathbb{E}_{a_{1:n}\sim\pi_\theta^k(\cdot|s)}\left[f(s,a_{1:n})\sum_{i=1}^n 1\{a_i = a\}\right]\right) \\ = & \alpha n \mathbb{E}[f(s,a_{1:n})]\mathrm{Cov}_{a\sim\pi_\theta^k(\cdot|s)}\left(\log \pi_\theta^k(a \mid s), \pi_\theta^k(a \mid s)\right) \\ & - \alpha \mathrm{Cov}_{a\sim\pi_\theta^k(\cdot|s)}\left(\log \pi_\theta^k(a \mid s), \mathrm{Cov}_{a_{1:n}}\left(f(s,a_{1:n}), \sum_{i=1}^n 1\{a_i = a\}\right) + \right. \\ & \left. n\pi_\theta^k(a \mid s)\mathbb{E}_{a_{1:n}\sim\pi_\theta^k(\cdot|s)}\left[f(s,a_{1:n})\right]\right) \\ = & -\alpha \mathrm{Cov}_{a\sim\pi_\theta^k(\cdot|s)}\left(\log \pi_\theta^k(a \mid s), \mathrm{Cov}_{a_{1:n}}\left(f(s,a_{1:n}), \sum_{i=1}^n 1\{a_i = a\}\right)\right).\end{aligned}$$

All that remains to show is

$$\begin{aligned}& \mathrm{Cov}_{a_{1:n}}\left(\frac{1}{n}\sum_{i=1}^n \log \pi_\theta^k(a_i \mid s), \mathrm{Cov}_{a'_{1:n}}\left(f(s,a'_{1:n}), \sum_{i,j=1}^n 1\{a'_i = a_j\}\right)\right) \\ & = \mathrm{Cov}_{a\sim\pi_\theta^k(\cdot|s)}\left(\log \pi_\theta^k(a \mid s), \mathrm{Cov}_{a_{1:n}}\left(f(s,a_{1:n}), \sum_{i=1}^n 1\{a_i = a\}\right)\right).\end{aligned}$$

This can be seen using the linearity of covariance and the fact that each a_i in $a_{1:n}$ is sampled independently:

$$\begin{aligned}& \mathrm{Cov}_{a_{1:n}}\left(\frac{1}{n}\sum_{i=1}^n \log \pi_\theta^k(a_i \mid s), \mathrm{Cov}_{a'_{1:n}}\left(f(s,a'_{1:n}), \sum_{i,j=1}^n 1\{a_i = a'_j\}\right)\right) \\ & = \frac{1}{n}\sum_{i=1}^n \mathrm{Cov}_{a_{1:n}}\left(\log \pi_\theta^k(a_i \mid s), \mathrm{Cov}_{a'_{1:n}}\left(f(s,a'_{1:n}), \sum_{j=1}^n 1\{a_i = a'_j\}\right)\right) \\ & = \mathrm{Cov}_{a\sim\pi_\theta^k(\cdot|s)}\left(\log \pi_\theta^k(a \mid s), \mathrm{Cov}_{a_{1:n}}\left(f(s,a_{1:n}), \sum_{i=1}^n 1\{a_i = a\}\right)\right).\end{aligned}$$

□

C.3 Proof of Lemma 5.3

Lemma 5.3 (restated). Consider any set of actions $a_{1:n} \in \mathcal{A}^n$. The change in the log probability of sampling this set of actions after one policy update using set RL can be written as the following first-order approximation:

$$\log \pi_\theta^{k+1}(a_{1:n} \mid s) \approx \log \pi_\theta^k(a_{1:n} \mid s) + \lambda \Lambda_f(a_{1:n}; \pi_\theta^k) - \lambda' C(\theta^k)$$

where $C(\theta^k)$ is a function independent of $a_{1:n}$.

Proof. This follows from using a first-order Taylor approximation of $\sum_{i=1}^n \log \pi_\theta^{k+1}(a_i \mid s)$ at $\sum_{i=1}^n \log \pi_\theta^k(a_i \mid s)$ and then using Lemma C.2. Proceeding this way, we get that,

$$C(\theta^k) = \mathrm{Cov}_{a'_{1:n} \sim \pi_\theta^k(\cdot|s)} \left(f(s, a'_{1:n}), \sum_{i=1}^n \pi_\theta^k(a'_i \mid s) \right).$$

This derivation gives us that $\lambda = \alpha I(a_{1:n})$ and $\lambda' = \alpha n$. □

C.4 Proof of Proposition 5.4

Proposition 5.4 (restated). Consider the polychromic objective in Eq. (7). For any homogeneous set $a_{1:n} = \{a\}$ where $r(s,a) = 1$, there exists $\epsilon \in (0,1)$ such that $\Lambda_{f_{\text{poly}}}(a) < 0$ when $\pi_\theta(a \mid s) > \epsilon$. Furthermore, the scaffold values of these homogeneous sets satisfy the bound $\Lambda_{f_{\text{poly}}}(a) \leq \sqrt{\frac{p(1-p)}{n}}$.

Proof. We first prove the first part of the proposition. Let $p = \pi_\theta(a \mid s)$. We will use the shorthand $\Lambda(a) = \Lambda_{f_{\text{poly}}}(a; \pi_\theta)$, $f = f_{\text{poly}}$ and $\hat{f} = \mathbb{E}_{a_{1:n} \sim \pi_\theta(\cdot|s)}[f(s, a_{1:n})]$. Then,

$$\begin{aligned}
\Lambda(a) &= \mathrm{Cov}_{a'_{1:n} \sim \pi_\theta(\cdot|s)}(\hat{f}(s, a'_{1:n}), \frac{1}{I(a)} \sum_{i,j=1}^n 1\{a'_i = a_j\}) \\
&= \mathrm{Cov}_{a'_{1:n} \sim \pi_\theta(\cdot|s)}(\hat{f}(s, a'_{1:n}), \frac{1}{n} \sum_{i=1}^n 1\{a'_i = a\}) \\
&= \sum_{j=0}^n \left(\sum_{|a'_{1:n} \cap \{a\}| = j} \pi_\theta(a'_{1:n} \mid s)(f(s, a'_{1:n}) - \hat{f})(\frac{j}{n} - \mathbb{E}_{\alpha_{1:n} \sim \pi_\theta(\cdot|s)} \left[\frac{1}{n} \sum_{i=1}^n 1\{\alpha_i = a\} \right]) \right) \\
&= \sum_{j=0}^n \left(\sum_{|a'_{1:n} \cap \{a\}| = j} \pi_\theta(a'_{1:n} \mid s)(f(s, a'_{1:n}) - \hat{f})(\frac{j}{n} - p) \right) \\
&= \sum_{j=0}^{\lfloor np \rfloor} (\frac{j}{n} - p) \left(\sum_{|a'_{1:n} \cap \{a\}| = j} \pi_\theta(a'_{1:n} \mid s)(f(s, a'_{1:n}) - \hat{f}) \right) \\
&+ \sum_{j=\lfloor np \rfloor + 1}^n (\frac{j}{n} - p) \left(\sum_{|a'_{1:n} \cap \{a\}| = j} \pi_\theta(a'_{1:n} \mid s)(f(s, a'_{1:n}) - \hat{f}) \right) \\
&= \sum_{j=0}^{\lfloor np \rfloor} (\frac{j}{n} - p) \left(\sum_{|a'_{1:n} \cap \{a\}| = j} \pi_\theta(a'_{1:n} \mid s)(f(s, a'_{1:n})) \right) \\
&+ \sum_{j=\lfloor np \rfloor + 1}^n (\frac{j}{n} - p) \left(\sum_{|a'_{1:n} \cap \{a\}| = j} \pi_\theta(a'_{1:n} \mid s)(f(s, a'_{1:n})) \right) \\
&- \hat{f} \left(\sum_{j=0}^n \sum_{|a'_{1:n} \cap \{a\}| = j} \pi_\theta(a'_{1:n} \mid s)(\frac{j}{n} - p) \right)
\end{aligned}$$

Now, we can simplify using properties of the binomial distribution as follows:

$$\sum_{j=0}^{n} \sum_{|a'_{1:n} \cap \{a\}|=j} \pi_\theta(a'_{1:n} \mid s)(\frac{j}{n} - p) = \sum_{j=0}^{n} \left(\frac{j}{n} - p\right) \binom{n}{j} p^j (1-p)^{n-j} = 0.$$

Therefore, the scaffold value becomes:

$$\begin{aligned}
\Lambda(a) &= \sum_{j=0}^{\lfloor np \rfloor} (\frac{j}{n} - p) \left(\sum_{|a'_{1:n} \cap \{a\}|=j} \pi_\theta(a'_{1:n} \mid s)(f(s, a'_{1:n})) \right) \\
&+ \sum_{j=\lfloor np \rfloor + 1}^{n} (\frac{j}{n} - p) \left(\sum_{|a'_{1:n} \cap \{a\}|=j} \pi_\theta(a'_{1:n} \mid s)(f(s, a'_{1:n})) \right) \\
&\leq \sum_{j=0}^{\lfloor np \rfloor} (\frac{j}{n} - p) \frac{2j}{n^2} \sum_{|a'_{1:n} \cap \{a\}|=j} \pi_\theta(a'_{1:n} \mid s) \\
&+ \sum_{j=\lfloor np \rfloor + 1}^{n} (\frac{j}{n} - p) \frac{n-j+1}{n} \sum_{|a'_{1:n} \cap \{a\}|=j} \pi_\theta(a'_{1:n} \mid s) \\
&= \sum_{j=0}^{\lfloor np \rfloor} (\frac{j}{n} - p) \frac{2j}{n^2} \binom{n}{j} p^j (1-p)^{n-j} \\
&+ \sum_{j=\lfloor np \rfloor + 1}^{n} (\frac{j}{n} - p) \frac{n-j+1}{n} \binom{n}{j} p^j (1-p)^{n-j} \\
&= \sum_{j=0}^{\lfloor np \rfloor} (\frac{j}{n} - p) \frac{2j}{n^2} \binom{n}{j} p^j (1-p)^{n-j} \\
&+ \sum_{j=\lfloor np \rfloor + 1}^{n-1} (\frac{j}{n} - p) \frac{n-j+1}{n} \binom{n}{j} p^j (1-p)^{n-j}
\end{aligned}$$

Here, in the second line, we used the fact when $a'_{1:n} \cap \{a\}$ is of size j, then the smallest possible value of $f(s, a'_{1:n})$ is $\frac{2j}{n^2}$ since at least 2 out of n elements are unique and at least j out of n elements attain reward $+1$. On the other hand, we can bound it above by $\frac{n-j+1}{n}$ which happens if all elements get reward $+1$ and $n-j+1$ elements are unique. In the last line, we used the fact that when $f(s, a'_{1:n} = \{a\}) = 0$ as the diversity is 0. Now, let $\epsilon = \frac{n-1}{n}$. Then, when $p \geq \epsilon$, $\lfloor np \rfloor \geq n-1$. Therefore,

$$\Lambda(a) = \sum_{j=0}^{\lfloor np \rfloor} (\frac{j}{n} - p) \frac{2j}{n^2} \binom{n}{j} p^j (1-p)^{n-j} \leq 0.$$

Now we prove the second part. First, define the following upper bound of the scaffold value of the homogeneous sets:

$$\Lambda(a) \leq \mathbb{E}_{X \sim \mathrm{Bin}(n,p)} \left[\left(\frac{X}{n} - p \right) C_n(X) \right] =: B_a(n)$$

where $C_n(x) = \begin{cases} \frac{2x}{n^2} & x \leq \lfloor np \rfloor \\ \frac{n-x+1}{n} & x \in [\lfloor np \rfloor + 1, n-1] \\ 0, & x = n \end{cases}$.

Now, $\mathbb{E}[\frac{X}{n} - p] = 0$ and $\mathbb{E}[(\frac{X}{n} - p)^2] = \mathrm{Var}(\frac{X}{n}) = \frac{1}{n^2}\mathrm{Var}(X) = \frac{p(1-p)}{n}$. On the other hand, we can bound $C_n(X)$ as follows: when $x \leq \lfloor np \rfloor$, $C_n(x) = \frac{2x}{n^2} \leq \frac{2\lfloor np \rfloor}{n^2} \leq \frac{2}{n}$. On the other hand, when $j \in [\lfloor np \rfloor + 1, n-1]$, we have $C_n(x) = \frac{n-x+1}{n} \leq 1$. Combining, we have that $\mathbb{E}[C_n(X)^2] \leq 1$. Therefore, using the Cauchy–Schwarz inequality:

$$\begin{aligned} B_a(n) &= \mathbb{E}_{X \sim \mathrm{Bin}(n,p)} \left[\left(\frac{X}{n} - p \right) C_n(X) \right] \\ &\leq \sqrt{\mathbb{E}\left[\left(\frac{X}{n} - p \right)^2 \right] \mathbb{E}\left[C_n(X)^2 \right]} \\ &\leq \sqrt{\frac{p(1-p)}{n}}. \end{aligned}$$

□

C.5 Proof of Proposition 5.5

Proposition 5.5 (restated). Suppose $a_{1:n}$ is heterogeneous where each a_i is unique with probability $\frac{p}{n}$ where $p \in (0, \frac{1}{n})$. Suppose exactly q of the n actions satisfy $r(s, a_i) = 1$, and that any other action $a' \notin a_{1:n}$ with $\pi_\theta(a' \mid s) > 0$ yields $r(s, a') = 0$. Then, the scaffold value of $a_{1:n}$ satisfies $\Lambda_{f_{\mathrm{poly}}}(a_{1:n}) > \frac{qp^n(1-p)}{n}\left(1 - \left(1 - \frac{1}{n}\right)^n\right)$.

Proof. This can be proven using very similar techniques. Let $P_j = \binom{n}{p} p^j (1-p)^{n-j}$. Ag ain, we use the shorthand $f = f_{\mathrm{poly}}$. Then,

$$\begin{aligned} \Lambda(a_{1:n}) &= \sum_{j=0}^{n} P_j (\frac{j}{n} - p) \mathbb{E}_{|a'_{1:n} \cap a_{1:n}| = j} \left[f(s, a'_{1:n}) \right] \\ &= P_0 (0 - p) \cdot 0 + \sum_{j=1}^{\lfloor np \rfloor} P_j (\frac{j}{n} - p) \mathbb{E}_{|a'_{1:n} \cap a_{1:n}| = j} \left[f(s, a'_{1:n}) \right] \\ &+ \sum_{j=\lfloor np \rfloor + 1}^{n} P_j (\frac{j}{n} - p) \mathbb{E}_{|a'_{1:n} \cap a_{1:n}| = j} \left[f(s, a'_{1:n}) \right] \\ &\geq P_n (\frac{n}{n} - p) \mathbb{E}_{|a'_{1:n} \cap a_{1:n}| = n} \left[f(s, a'_{1:n}) \right] \\ &= P_n (1 - p) \mathbb{E}_{|a'_{1:n} \cap a_{1:n}| = n} \left[f(s, a'_{1:n}) \right] \end{aligned}$$

All that remains to show is that

$$\mathbb{E}_{|a'_{1:n} \cap a_{1:n}| = n} \left[f(s, a'_{1:n}) \right] = \frac{q}{n} \left(1 - \left(1 - \frac{1}{n} \right)^n \right).$$

Note that, since the intersection's cardinality is n and the sampling is done independently, this is equivalent to drawing n items uniformly with replacement from $a_{1:n}$. Also note that $\mathbb{E}\left[\frac{1}{n}\sum_i r(s, a'_i) \cdot d(s, a'_{1:n})\right] = \frac{1}{n}\sum_i \mathbb{E}[r(s, a'_i) \cdot d(s, a'_{1:n})]$. Since this is a uniform distribution, this becomes equal to $\mathbb{E}[r(s, a'_i)]\mathbb{E}[d(s, a'_{1:n})] = \frac{q}{n}\mathbb{E}[d(s, a'_{1:n})]$.

On the other hand, to compute $\mathbb{E}[d(s, a'_{1:n})]$, we use indicator variables. Let $1_k = 1\{a_k \in a'_{1:n}\}$. Then, $\mathbb{E}\left[1_k\right] = 1 - \left(1 - \frac{1}{n}\right)^n$. Now,

$$\begin{aligned} \mathbb{E}_{|a'_{1:n} \cap a_{1:n}| = n}[d(s, a'_{1:n})] &= \mathbb{E}_{|a'_{1:n} \cap a_{1:n}| = n}[\frac{1}{n} \sum_k 1_k] \\ &= 1 - \left(1 - \frac{1}{n} \right)^n \end{aligned}$$

□